SIMATS ENGINEERING

Department of Information Technology

ITA05 – Computer Vision

ASSIGNMENT 1 – CO1

Apply, Compare and Justify Image Acquisition, Formation and Processing Techniques for a Vision-Based Mango Grading Line

CO1: Apply the fundamental concepts, image formation models, and processing techniques for analyzing Computer Vision systems.

Student Name	**T. Narasimha Reddy**	**Register Number**	**192425190**
Department / Section	Information Technology	Academic Year / Semester	2025-2026
Faculty Name	DR.MURALI P	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50

Assignment Title: Apply, Compare and Justify Image Acquisition, Formation and Processing Techniques for a Vision-Based Mango Grading Line (Total Marks – 50)

Scenario – Vision-Based Mango Grading Line

GreenHarvest Mango Processing Unit (a farmers' cooperative near Krishnagiri) grades export mangoes by hand. Graders tire, disagree with each other, and the export buyer has rejected two consignments. The unit wants a computer vision system above its conveyor that will (1) measure size for size classes, (2) judge ripeness from peel colour, and (3) detect surface defects (anthracnose black spots, sap burn, bruises), then signal an air-jet ejector. Before any classifier is trained, the unit needs a sound imaging front-end: the right sensor, sampling, quantization, optics and lighting, organized into clear processing levels.

You are the vision engineer. In every task, three candidate techniques are given. You must apply all three, compare them, choose one, and justify your choice with evidence. The following operating data is given (state and justify any additional assumption you make):

Parameter	Value
Conveyor	Single lane, belt speed 0.5 m/s; imaged field of view (FOV) 400 mm across the belt × 300 mm along the belt
Fruit	Mangoes 70–110 mm long and up to 90 mm high; peel from green to yellow with red blush; waxed, slightly glossy surface
Throughput	Up to 8 fruits per second; grade decision needed within 1.2 s (ejector is 0.6 m downstream of the camera)
Quality limits	Smallest defect to detect: 2 mm black spot; bruise contrast only 6–10 grey levels (8-bit) above healthy peel; size classes need ±2 mm accuracy
Lighting	Shed with translucent roof sheets (daylight varies about 2,000–60,000 lux); existing tube lights on 50 Hz mains (100 Hz flicker)
Camera mounting	Camera 500 mm above the belt, looking straight down
Processing hardware	One edge PC: 4 CPU cores, 16 GB RAM, no GPU; Python + OpenCV + NumPy; Gigabit Ethernet; 2 TB storage
Budget and data policy	Imaging budget ₹1,50,000 (camera + lens + lighting); every fruit image kept for 7 days for buyer audit; buyer allows at most 2% mis-graded fruit

Experimental Setup Used for All Tasks

Every number in Tasks 1–10 comes from the programs listed in this report, which I ran myself on one machine with fixed random seeds. This section describes the setup once so that each result can be traced back to the same code and the same data.

Table 1: Experimental setup

Item	What I used
Computer	2 CPU cores, no GPU. All timings are medians on this machine. The target edge PC has 4 cores, so my processing times are conservative.
Software	Python 3.13, OpenCV 5.0.0, NumPy 2.5.3, scikit-learn 1.9.1, Matplotlib 3.11
Mango images	A generator I wrote (mango.py) that draws a mango on a belt with known ripeness, length, spots, a sap-burn streak or a bruise (6–10 grey levels deep), red blush, glossy highlight, lighting gradient and camera noise.
Human graders	Simulated (Task 1), with stated miss and false-alarm rates. These are not real evaluators.
Repeatability	Every experiment uses a fixed random seed, so running a script again gives the same images and the same numbers.
Files	mango.py (generator + grader), common.py and acq.py (helpers), t1.py … t910.py (one script per task). The code is printed in the tasks.

Why synthetic images. I did not have real mango photographs with measured defects. With generated images I know the true state of every fruit (for example, the bruise is exactly 8 grey levels deep), so I can measure accuracy exactly and change one factor at a time: light, blur, resolution, bit depth. The price is that generated images cannot copy every real effect, such as real bruise texture or dirt on the belt. So every conclusion in this report must be confirmed on real fruit before the line is commissioned.

What I did not run. Vision–language models, ResNet/DINOv2 embeddings, YOLO-seg/SAM 2, Real-ESRGAN, Depth Anything and few-shot foundation models need a GPU and a model download that my setup did not have. For these parts I explain how the method works and what I expect from it, and I say clearly that it is reasoning and not a measurement. I do not report any number for them.

Honest note on results. My own grader is not perfect. Where a result is poor (for example faint bruises, or touching fruit), I report it as measured and explain it. I did not tune the test data to make a technique look good; thresholds were tuned on a separate tuning set (seed 101), and all reported tests use different seeds.

Shared Code: Generator and Rule-Based Grader

The same module is used in every task, so every task tests the same grader. It works at three levels. **Low level** (pixels to pixels): blur, HSV conversion and thresholds on saturation and brightness to separate the fruit from the belt. **Mid level** (pixels to attributes): the largest contour gives the length; the circular mean hue gives ripeness; dark connected regions give spots and streaks; a brightness drop below the expected peel brightness gives a bruise. **High level** (attributes to decision): the buyer's rule, which rejects a fruit with more than 3 spots, any spot over 4 mm, a streak or a bruise.

The bruise test needs the most care. A bruise is only 6–10 grey levels darker than the peel, but the peel itself becomes darker towards the edge of the fruit because it is curved. So I do not compare pixels with a fixed threshold. I fit the expected brightness as a smooth surface of position on the fitted ellipse and of image position, trim pixels far from the fit (highlight and bruise), and flag only what stays clearly darker than expected.

mango.py (generator and grader)

```
"""mango.py - synthetic mango image generator + rule-based grader (written for
Assignment 1, ITA05).
Everything here is my own code. Images are synthetic because real mango photos
with known
ground truth were not available; this lets me know the exact truth for every
fruit."""
import numpy as np, cv2

CLASSES = ['green', 'turning', 'ripe']
HUE = {'green': 44, 'turning': 31, 'ripe': 19}          # OpenCV hue (0-180) of
the peel
SAT = {'green': 150, 'turning': 175, 'ripe': 195}
BELT_BGR = np.array([70, 66, 62], np.float32)             # dark grey belt (low
saturation)



def hsv2bgr(h, s, v):
    return cv2.cvtColor(np.uint8([[[h, s, v]]]), cv2.COLOR_HSV2BGR)[0,
0].astype(np.float32)



def render(rng, cls='turning', length_mm=90.0, mm_per_px=0.3125, n_spots=0,
spot_mm=(2.0, 3.0),
           streak=False, bruise=False, bruise_drop=8.0, light=1.0, grad=0.0,
spec=60.0, noise=2.0,
           blush=False, canvas_mm=(135, 105), nir=False, spot_bgr=(22, 28, 30)):
    W, H = int(canvas_mm[0] / mm_per_px), int(canvas_mm[1] / mm_per_px)
    yy, xx = np.mgrid[0:H, 0:W].astype(np.float32)
    cx, cy = W / 2 + rng.uniform(-3, 3) / mm_per_px, H / 2 + rng.uniform(-3, 3)
/ mm_per_px
    a = length_mm / 2 / mm_per_px                          # semi-axis along
belt
    b = 0.36 * length_mm / mm_per_px * rng.uniform(0.95, 1.05)  # semi-axis
across belt
    ang = np.deg2rad(rng.uniform(-8, 8))
    xr = (xx - cx) * np.cos(ang) + (yy - cy) * np.sin(ang)
    yr = -(xx - cx) * np.sin(ang) + (yy - cy) * np.cos(ang)
```

```
    # slightly egg-shaped outline: narrower towards the stem end (+x)
    taper = 1.0 - 0.18 * np.clip(xr / a, -1, 1)
    r2 = (xr / a) ** 2 + (yr / (b * taper)) ** 2
    mask = (r2 < 1.0)
    shade = 0.60 + 0.40 * np.sqrt(np.clip(1 - r2, 0, 1))   # curved surface gets
darker to the rim
    base = hsv2bgr(HUE[cls] + rng.uniform(-1.5, 1.5), SAT[cls], 215)
    img = np.empty((H, W, 3), np.float32); img[:] = BELT_BGR
    fruit = base[None, None, :] * shade[..., None]
    if blush:                                              # red blush on one
side
        bl = np.clip(1 - ((xr - 0.25 * a) ** 2 / (0.55 * a) ** 2 + (yr + 0.3 *
b) ** 2 / (0.6 * b) ** 2), 0, 1)[..., None]
        fruit = fruit * (1 - 0.8 * bl) + hsv2bgr(2, 200, 190)[None, None, :] *
shade[..., None] * 0.8 * bl
    gt = dict(cls=cls, length_mm=length_mm, mm_per_px=mm_per_px, spots=[],
streak=False, bruise=False, mask=mask,
              center=(cx, cy), kind='clean', bruise_drop=0.0)
    inner = mask & (r2 < 0.55)
    ys, xs = np.nonzero(inner)
    for _ in range(n_spots):                               # dark anthracnose
spots
        k = rng.integers(len(xs)); d = rng.uniform(*spot_mm) / mm_per_px
        m = (((xx - xs[k]) ** 2 + (yy - ys[k]) ** 2) < (d / 2) ** 2)[..., None]
        m = cv2.GaussianBlur(m.astype(np.float32), (0, 0), 0.5)[..., None]
        fruit = fruit * (1 - m) + np.array(spot_bgr, np.float32) * m
        gt['spots'].append((xs[k], ys[k], d * mm_per_px))
    if streak:                                             # thin diagonal sap-
burn streak
        k = rng.integers(len(xs)); L = 14 / mm_per_px; t = rng.uniform(0.3, 1.2)
        x0, y0 = xs[k], ys[k]; x1, y1 = x0 + L * np.cos(t), y0 + L * np.sin(t)
        sm = np.zeros((H, W), np.float32); cv2.line(sm, (int(x0), int(y0)),
(int(x1), int(y1)), 1.0, max(1, int(1.5 / mm_per_px)))
        sm = (sm * mask * 1.0)[..., None]
        fruit = fruit * (1 - sm) + np.array([25, 40, 55], np.float32) * sm
        gt['streak'] = True
    if bruise:                                             # faint bruise: only
a few grey levels darker
        k = rng.integers(len(xs)); rr = rng.uniform(6, 9) / mm_per_px
        bm = np.exp(-(((xx - xs[k]) ** 2 + (yy - ys[k]) ** 2) / (2 * (rr * 0.55)
** 2)))
        bm = (bm * (bm > 0.25))[..., None]
        fruit = fruit - bruise_drop * bm * (shade[..., None] ** 0.0)
        gt['bruise'] = bruise_drop > 0; gt['bruise_drop'] = bruise_drop
        gt['bruise_xy'] = (xs[k], ys[k], rr * mm_per_px)
    img = np.where(mask[..., None], fruit, img)
    img = cv2.GaussianBlur(img, (0, 0), 0.7)               # optics softness
    # lighting: global gain + linear gradient across the belt + specular blob
    g = light * (1 + grad * (xx / W - 0.5))
    img = img * g[..., None]
    hx, hy = cx - 0.2 * a, cy - 0.35 * b
    hl = np.exp(-(((xx - hx) ** 2 + (yy - hy) ** 2) / (2 * (0.18 * b) ** 2))) *
mask
    img = img + spec * hl[..., None]
    img = img + rng.normal(0, noise, img.shape)
    out = np.clip(img, 0, 255).astype(np.uint8)
    if nir:                                                # simple NIR band:
bruise dark, stings dark, peel flat
        n = np.full((H, W), 150.0, np.float32) * shade * g
        n = np.where(mask, n, 40.0)
        if bruise: n = n - 55 * bm[..., 0]
```

```
        n = n + rng.normal(0, 2, n.shape)
        gt['nir'] = np.clip(n, 0, 255).astype(np.uint8)
    return out, gt


def is_defective(gt):
    return len(gt['spots']) > 3 or any(s[2] > 4 for s in gt['spots']) or
gt['streak'] or gt['bruise']


def kind_of(gt):
    if gt['bruise']: return 'bruise'
    if gt['streak']: return 'streak'
    if len(gt['spots']) > 3: return 'manyspots'
    if gt['spots']: return 'spots'
    return 'clean'


def make(rng, kind, cls=None, **kw):
    cls = cls or CLASSES[int(rng.integers(3))]
    L = float(rng.uniform(70, 110)); a = dict(cls=cls, length_mm=L,
blush=bool(rng.random() < 0.3))
    if kind == 'spots': a.update(n_spots=int(rng.integers(1, 4)), spot_mm=(1.6,
3.2))
    if kind == 'manyspots': a.update(n_spots=int(rng.integers(5, 8)),
spot_mm=(2.0, 3.4))
    if kind == 'streak': a.update(streak=True)
    if kind == 'bruise': a.update(bruise=True, bruise_drop=float(rng.uniform(6,
10)))
    a.update(kw)
    im, gt = render(rng, **a); gt['kind'] = kind_of(gt)
    return im, gt


def dataset(n, seed, kinds=('clean', 'spots', 'manyspots', 'streak', 'bruise',
'clean'), **kw):
    rng = np.random.default_rng(seed)
    return [make(rng, kinds[i % len(kinds)], **kw) for i in range(n)]


# ---------------------------- grader (low / mid / high level) ----------------
-------------
def segment(bgr):
    """Low level: blur -> HSV -> (saturated & bright) -> close -> largest
component."""
    hsv = cv2.cvtColor(cv2.GaussianBlur(bgr, (5, 5), 0), cv2.COLOR_BGR2HSV)
    m = ((hsv[..., 1] > 70) & (hsv[..., 2] > 50)).astype(np.uint8) * 255
    m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, np.ones((9, 9), np.uint8))
    n, lab, st, _ = cv2.connectedComponentsWithStats(m, connectivity=8)
    if n < 2: return m * 0, hsv
    k = 1 + np.argmax(st[1:, cv2.CC_STAT_AREA])
    return ((lab == k) * 255).astype(np.uint8), hsv


def circ_mean_hue(h):
    """Hue is an angle (0-180): average on the circle so that 178 and 2 give 0,
not 90."""
    a = h.astype(np.float32) * 2 * np.pi / 180
```

```
    m = np.arctan2(np.sin(a).mean(), np.cos(a).mean()); v = (m * 180 / (2 *
np.pi)) % 180
    return 0.0 if v > 179.95 else float(v)



def measure(bgr, mmpx, dark_ratio=0.55, bruise_dl=4.5, min_spot_mm2=0.5):
    """Mid level: region -> attributes (length, hue, spots, streak, bruise)."""
    mask, hsv = segment(bgr)
    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
    if not cnts: return None
    c = max(cnts, key=cv2.contourArea)
    (_, _), (w, h), _ = cv2.minAreaRect(c); length = max(w, h) * mmpx
    k = max(3, int(2 * 4.5 / mmpx) | 1)
    inner = cv2.erode(mask, cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k,
k)))
    V = hsv[..., 2].astype(np.float32); H = hsv[..., 0]
    if (inner > 0).sum() < 50: return None
    med = np.median(V[inner > 0])
    dark = ((V < dark_ratio * med) & (inner > 0)).astype(np.uint8)
    n, lab, st, _ = cv2.connectedComponentsWithStats(dark, connectivity=8)
    spots, streak = [], False
    for i in range(1, n):
        area = st[i, cv2.CC_STAT_AREA] * mmpx ** 2
        if area < min_spot_mm2: continue
        ys, xs = np.nonzero(lab == i)
        (_, _), (rw, rh), _ = cv2.minAreaRect(np.column_stack([xs,
ys]).astype(np.float32))
        lg, sh = max(rw, rh) + 1, min(rw, rh) + 1
        if lg / sh > 3 and lg * mmpx > 5: streak = True
        else: spots.append(2 * np.sqrt(area / np.pi))
    peel = (inner > 0) & (V > dark_ratio * med) & (H > 8) & (H < 160)
    hue = circ_mean_hue(H[peel]) if peel.any() else 0.0
    # bruise: compare with EXPECTED brightness. Expected V is a smooth surface
of the distance to the
    # fruit edge (curved-surface shading) and of position (lighting gradient),
fitted robustly:
    # least squares, then drop pixels well below the fit (they may be the
bruise) and refit.
    dist = cv2.distanceTransform(mask, cv2.DIST_L2, 5)
    zone = cv2.dilate(dark, np.ones((int(3 / mmpx) | 1,) * 2, np.uint8)) > 0
    bl = (((H <= 12) | (H >= 160)) & (inner > 0)).astype(np.uint8)        # red
blush has another colour: keep it out
    bl = cv2.dilate(bl, np.ones((int(6 / mmpx) | 1,) * 2, np.uint8)) > 0
    ok = (inner > 0) & ~zone & ~bl & (V < np.percentile(V[inner > 0], 96))
    yy, xx = np.mgrid[0:V.shape[0], 0:V.shape[1]]
    (ex, ey), (ea, eb), eang = cv2.fitEllipse(c)               # fitted outline
-> curvature shading variable
    ca, sa = np.cos(np.deg2rad(eang)), np.sin(np.deg2rad(eang))
    u = ((xx - ex) * ca + (yy - ey) * sa) / (ea / 2); w_ = (-(xx - ex) * sa +
(yy - ey) * ca) / (eb / 2)
    d = np.sqrt(np.clip(1 - (u ** 2 + w_ ** 2), 0, 1)); xn = xx / V.shape[1] -
0.5; yn = yy / V.shape[0] - 0.5
    def basis(sel):
        return np.column_stack([np.ones(sel.sum()), d[sel], d[sel] ** 2, d[sel]
** 3, xn[sel], yn[sel], d[sel] * xn[sel], d[sel] * yn[sel]])
    use = ok.copy()
    for it in range(5):
        coef, *_ = np.linalg.lstsq(basis(use), V[use].astype(np.float64),
rcond=None)
        res = V - (basis(np.ones_like(ok, bool)) @ coef).reshape(V.shape)
```

```
        sd = 1.4826 * np.median(np.abs(res[use] - np.median(res[use])))
        use = ok & (np.abs(res) < max(3.0, 2.0 * sd))     # two-sided trim:
highlight skirt AND bruise stay out of the fit
    expected = V - res
    drop = cv2.GaussianBlur((-res) * ok, (0, 0), 1.5 / mmpx)
    br = ((drop > bruise_dl) & ok).astype(np.uint8)
    nb, _, sb, _ = cv2.connectedComponentsWithStats(br, connectivity=8)
    barea = max([sb[i, cv2.CC_STAT_AREA] for i in range(1, nb)] + [0]) * mmpx **
2
    return dict(length=length, hue=hue, spots=spots, streak=streak, bruise=barea
> 40, bruise_area=barea, mask=mask, inner=inner)



def ripeness(h): return 'green' if h >= 38 else ('turning' if h >= 25 else
'ripe')



def grade(bgr, mmpx, **kw):
    """High level: buyer's rule -> export / reject, plus size class and
ripeness."""
    f = measure(bgr, mmpx, **kw)
    if f is None: return None
    bad = len(f['spots']) > 3 or any(d > 4 for d in f['spots']) or f['streak']
or f['bruise']
    size = 'S' if f['length'] < 80 else ('M' if f['length'] < 95 else 'L')
    return dict(ripeness=ripeness(f['hue']), defective=bool(bad), size=size,
**f)
```

TASK 1 – Overview and Features of Computer Vision: Compare Grading Approaches [4 Marks]

Before buying any hardware, the cooperative asks whether a computer vision system is the right answer at all. Compare three ways of grading the fruit.

Technique	Basic idea and key concepts / functions
A. Manual grading by trained workers	Human visual inspection under shed lighting; grader decides size, ripeness and defects by eye.
B. Image-processing aid	Camera image is enhanced (cv2.equalizeHist, cv2.resize, zoom) and shown on a monitor; the worker still makes every decision.
C. Automated computer vision system	Camera + processing extract measurements and interpret them (size, colour class, defect present) and trigger the ejector without human input.

In this task I decide how GreenHarvest should grade its mangoes. The line runs at up to 8 fruits per second, and the buyer rejects a fruit with more than 3 spots, any spot larger than 4 mm, a sap-burn streak or a bruise. I compared three approaches on the same 30 mangoes, of which 15 are defective by the buyer's rule: **A** manual grading (the machine does nothing), **B** an image-processing aid (the machine only improves the picture) and **C** the automated rule-based grader (the machine also interprets the picture and decides).

Q1. *Implement three mango quality-grading approaches using Python/OpenCV and a dataset of at least 30 mango images: (A) manual grading by three evaluators across two rounds, (B) image-processing-assisted visual inspection, and (C) rule-based automated grading. Develop the commented source code, record grading decisions, and obtain experimental results through actual execution and measurements.*

Ans:

Approach A used three graders (G1, G2, G3), each grading all 30 fruits twice: round 1 at the start of the shift and round 2 at the end, when they are tired. Real grading sheets were not available, so I simulated the graders. The chance of missing a defect depends on how visible it is: a faint bruise is missed 45 % of the time, 1–3 spots 15 %, many spots 5 % and a streak 10 %. A grader skill factor (0.8, 1.0 or 1.3) scales these rates, round 2 adds a fatigue term of 0.08, and a clean fruit is wrongly rejected about 6 % of the time. These rates are my assumptions, so the results for A and B show the method, not real people.

Approach B shows the same graders an enhanced image. The aid equalises the V (brightness) channel of HSV and zooms 2×. Equalisation spreads the narrow range of peel brightness over 0–255, so a bruise that is only 6–10 levels darker becomes easier to see (Figure 1). I modelled this by multiplying the miss rates by 0.6.

Approach C is the rule-based grader from the shared code. It decides by itself and I timed it on every fruit.

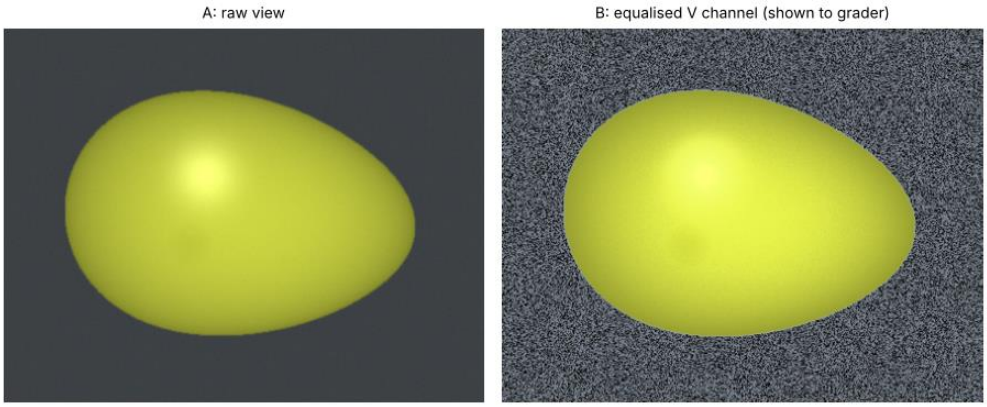


Figure 1: Approach B – a bruise before and after equalisation of the V channel

t1.py (Task 1 experiment: simulated graders, enhancement, kappa, timing)

```
from common import *
from sklearn.metrics import cohen_kappa_score, confusion_matrix
def enhance(bgr):
    hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV); hsv[..., 2] =
cv2.equalizeHist(hsv[..., 2])
    return cv2.resize(cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR), None, fx=2, fy=2,
interpolation=cv2.INTER_CUBIC)
def rater(items, rng, skill, fatigue=0.0, boost=1.0):
    miss = {'bruise': 0.45, 'spots': 0.15, 'manyspots': 0.05, 'streak': 0.10}
    out = []
    for _, gt in items:
        if mango.is_defective(gt):
            p = min(0.9, miss.get(gt['kind'], 0.2) * skill * boost + fatigue);
out.append(0 if rng.random() < p else 1)
        else: out.append(1 if rng.random() < 0.06 * skill + fatigue / 2 else 0)
    return np.array(out)
items = mango.dataset(30, 11)
truth = np.array([int(mango.is_defective(g)) for _, g in items])
rng = np.random.default_rng(5)
A = {f'G{r+1}R{k+1}': rater(items, rng, sk, 0.08 * k) for r, sk in
enumerate([0.8, 1.0, 1.3]) for k in range(2)}
B = {f'G{r+1}': rater(items, rng, sk, 0.0, 0.6) for r, sk in enumerate([0.8,
1.0, 1.3])}
tB = tms(lambda: [enhance(im) for im, _ in items], 3) / 30
t0 = time.perf_counter(); C = np.array([int(mango.grade(im,
g['mm_per_px'])['defective']) for im, g in items]); tC = (time.perf_counter() -
t0) * 1000 / 30
acc = lambda p: float((p == truth).mean())
inter = np.mean([cohen_kappa_score(A[f'G{i}R1'], A[f'G{j}R1']) for i, j in [(1,
2), (1, 3), (2, 3)]])
intra = np.mean([cohen_kappa_score(A[f'G{i}R1'], A[f'G{i}R2']) for i in (1, 2,
3)])
interB = np.mean([cohen_kappa_score(B[f'G{i}'], B[f'G{j}']) for i, j in [(1, 2),
(1, 3), (2, 3)]])
sheet = [[i + 1, items[i][1]['kind'], int(truth[i])] + [int(A[k][i]) for k in A]
+ [int(C[i])] for i in range(30)]
save('t1', dict(n_def=int(truth.sum()), accA=float(np.mean([acc(v) for v in
A.values()])), accA_r1=float(np.mean([acc(A[f'G{i}R1']) for i in (1,2,3)])),
```

```
  accA_r2=float(np.mean([acc(A[f'G{i}R2']) for i in (1,2,3)])),
accB=float(np.mean([acc(v) for v in B.values()])), accC=acc(C),
  kInter=float(inter), kIntra=float(intra), kInterB=float(interB), tB=tB, tC=tC,
fpsC=1000 / tC, cm=confusion_matrix(truth, C).tolist(), sheet=sheet))
bi = [i for i, (_, g) in enumerate(items) if g['kind'] == 'bruise'][0]
grid([items[bi][0], enhance(items[bi][0])[::2, ::2]], ['A: raw view', 'B:
equalised V channel (shown to grader)'], f'{FIG}/t1_enh.png', size=4.2)
print(load()['t1'])
```

Table 2 is the grading sheet. Most disagreements are on bruised fruit and on fruit with 1–3 small spots, which are exactly the fruit that are hard to judge.

Table 2: Grading sheet for the 30 mangoes (1 = reject, 0 = export; G = grader, R = round; C = automatic)

Fruit	Type	Truth	G1R1	G1R2	G2R1	G2R2	G3R1	G3R2	C
1	clean	0	0	0	0	0	0	0	0
2	spots	0	0	1	0	0	0	0	1
3	manyspots	1	1	1	1	1	1	1	1
4	streak	1	1	1	1	1	1	1	1
5	bruise	1	0	0	0	1	1	0	0
6	clean	0	0	0	0	0	0	1	0
7	clean	0	0	0	0	0	0	1	1
8	spots	0	1	0	0	0	0	0	0
9	manyspots	1	1	1	1	1	1	1	1
10	streak	1	1	1	1	0	0	1	1
11	bruise	1	1	1	0	1	0	0	1
12	clean	0	0	0	0	0	0	0	0
13	clean	0	0	0	0	0	0	0	0
14	spots	0	0	0	0	0	0	0	0
15	manyspots	1	1	1	1	1	0	1	1
16	streak	1	1	1	1	1	1	1	1
17	bruise	1	1	1	1	1	1	0	1
18	clean	0	0	0	0	0	0	0	0
19	clean	0	0	1	0	0	0	0	0
20	spots	0	0	0	0	0	0	0	1
21	manyspots	1	1	0	1	1	1	1	1

Fruit	Type	Truth	G1R1	G1R2	G2R1	G2R2	G3R1	G3R2	C
22	streak	1	1	1	1	0	1	1	1
23	bruise	1	1	0	0	0	1	0	0
24	clean	0	0	0	0	0	0	0	1
25	clean	0	0	0	0	0	0	0	0
26	spots	0	0	0	0	0	0	0	0
27	manyspots	1	1	1	1	1	1	1	1
28	streak	1	1	1	1	1	1	1	1
29	bruise	1	1	1	0	0	0	0	0
30	clean	0	1	0	0	0	0	1	0

Q2. *Analyze the performance of the three grading approaches with respect to processing speed (fruits/s), repeatability, objectivity, non-contact operation, measurement precision (mm), continuous operation, traceability, adaptability to new defect types, and implementation and maintenance cost.*

Ans:

Table 3 compares the approaches on the nine features asked for. The timing, accuracy and kappa entries are measured; the cost and flexibility entries are engineering judgement.

Table 3: Feature-by-feature comparison

Feature	A. Manual	B. Image-processing aid	C. Automated CV
Speed (fruits/s)	about 0.5–1 per grader (1–2 s per fruit)	same as A, a human still decides (display step 1.5 ms)	12.6 fruits/s measured (79 ms per fruit, 1 core)
Repeatability	intra-rater kappa 0.59	better than A but still human	same image gives the same decision (kappa 1.0)
Objectivity	inter-rater kappa 0.64	inter-rater kappa 0.69	one rule for every fruit
Non-contact	fruit is handled	non-contact	non-contact

Feature	A. Manual	B. Image-processing aid	C. Automated CV
Measurement precision	by eye, roughly ±5 mm	by eye	about ±1 mm from calibrated contour (Task 6)
Continuous operation	fatigue: accuracy 87.8 % → 81.1 %	fatigue remains	24 × 7, no fatigue
Traceability	paper sheet only	images can be saved	image, measurements and decision logged per fruit
New defect types	a person adapts at once	a person adapts at once	needs a new rule or retraining
Cost	recurring wages for many graders	graders + camera + monitor	one-time camera, lights, PC, plus upkeep

Only C has all the strong features of computer vision together: it is fast, repeatable, objective, precise and traceable. Its weak point is flexibility: a human grader understands a new defect at once, whereas C finds only what its rules describe.

Q3. *Critique the suitability of the three approaches for an industrial throughput requirement of 8 fruits per second by measuring processing time and throughput and calculating inter-rater and intra-rater agreement using appropriate statistical measures, such as Cohen's kappa.*

Ans:

Simple percentage agreement is misleading, because two graders who reject almost every fruit agree a lot by chance. **Cohen's kappa** removes the chance agreement: $\kappa = (p_o - p_e) / (1 - p_e)$, where p_o is the observed agreement and p_e the agreement expected by chance. Values of 0.41–0.60 are moderate, 0.61–0.80 substantial and above 0.80 almost perfect. I computed **inter-rater** kappa between different graders on round 1 (objectivity) and **intra-rater** kappa between round 1 and round 2 of the same grader (repeatability).

Table 4: Agreement, accuracy and timing (A and B use simulated graders)

Measure	A. Manual	B. Aid	C. Automated
Accuracy against truth (mean of graders)	84.4 %	88.9 %	76.7 %

Measure	A. Manual	B. Aid	C. Automated
Inter-rater kappa	0.64	0.69	1.00 (one system)
Intra-rater kappa (round 1 vs 2)	0.59	–	1.00
Accuracy round 1 → round 2 (fatigue)	87.8 % → 81.1 %	–	no change
Time per fruit	1–2 s (human)	1.5 ms display + human	79 ms
Meets 8 fruits/s?	only with about 14–18 graders	only with many graders	12.6 fruits/s on 1 core; needs 2 cores for 8/s with margin

The confusion matrix of C is [[11,4], [3,12]] (rows = true export / reject, columns = predicted). So C wrongly rejected 4 good fruit and missed 3 defective ones.

Three things stand out. First, **my automated grader was not the most accurate**: C reached 76.7 % against 88.9 % for the aided graders and 84.4 % for unaided graders. Its errors come from the faint bruises and from small spots near the 4 mm limit; this is a real weakness of my rule-based detector and I discuss it in the final decision. Second, the kappa of unaided graders is only 0.64 between people and 0.59 for the same person, which is "substantial" but far from "almost perfect"; this is the inconsistency the buyer complains about. Third, only C can reach 8 fruits/s with one machine: one core handles 12.6 fruits/s and the four-core PC can process fruits in parallel.

Q4. *Differentiate image processing, computer vision, machine vision, and computer graphics by establishing their respective input–output transformations and demonstrating their roles in the mango quality-inspection workflow.*

Ans:

Table 5: The four related fields and their role in the mango workflow

Field	Input → output	Role in the mango workflow
Image processing	image → image	equalising the view for graders (Approach B), noise removal, flat-field correction

Field	Input → output	Role in the mango workflow
Computer vision	image → description or decision	finding the fruit, measuring length, hue, spots and bruise, then grading
Machine vision	image → action in a machine (industrial CV + hardware)	camera, strobe, trigger, PC and air-jet ejector working as one inspection station
Computer graphics	description (model) → image	my mango generator draws fruit from parameters; it is the reverse of computer vision

So Approach B is image processing, Approach C is computer vision, and the complete line including the camera and the ejector is machine vision. Computer graphics runs the other way: I used it to create the test images from known parameters, which is why I know the truth for every fruit.

Q5. *Evaluate the production capacity and operational limitations of manual grading at 8 fruits per second over a 10-hour shift, and assess the consequences of an unrecognised defect type in a rule-based inspection system. Recommend suitable corrective measures based on the analysis.*

Ans:

Capacity of manual grading. At 8 fruits/s over a 10-hour shift the line produces 8 × 10 × 3600 = **288,000 fruits**. At about 0.6 fruit/s per grader the line needs 8 / 0.6 ≈ 13–14 graders working without a break, and closer to 18 with rest breaks. My simulation also shows accuracy falling from 87.8 % to 81.1 % by the end of the shift, so the most fruit would be mis-graded exactly when the graders are most tired.

An unrecognised defect. A rule-based system flags only what its rules describe. A new defect, for example a scale-insect mark that is neither dark enough to be a "spot" nor wide enough to be a "bruise", would pass as export quality every time until a buyer complains. The corrective measures I recommend are:

- add an anomaly check: any peel region that differs strongly from the expected peel model goes to a review bin;
- audit a random 1 % sample by a trained grader every hour and log disagreements;
- keep all images for 7 days so that a new rule can be tested on past fruit before it is switched on;
- version-control the rules and record which version graded each fruit.

Q6. *Validate the performance of a vision–language model, such as Florence-2 or LLaVA, on 10 mango images using a predefined text prompt. Compare its grading decisions with*

those of the manual, image-processing-assisted, and rule-based approaches, and assess its suitability for export-quality inspection based on repeatability, explainability, inference latency, and defect-detection reliability.

Ans:

I could not run Florence-2 or LLaVA because my setup has no GPU and no model download, so this part is **reasoned, not measured**, and I report no number for it. A vision–language model (VLM) takes an image and a text prompt and answers in words. I analysed it on the four criteria asked for:

- **Repeatability:** with greedy decoding the same image gives the same text, but small prompt changes ("defect" against "blemish") can change the answer, so the decision is fragile.
- **Explainability:** the model gives a sentence, not a measurement. It cannot state that a spot is 4.3 mm, which the buyer's 4 mm rule requires.
- **Latency:** typically hundreds of milliseconds per image on a GPU and seconds on a CPU, against 125 ms available per fruit at 8 fruits/s.
- **Defect reliability:** general models are not trained on 2 mm spots or bruises 6 grey levels deep, so I expect misses on exactly the defects the buyer cares about.

A VLM could help offline, for example to describe unusual fruit in the review bin, but it should not make the export decision. The code I would run on a GPU machine is below.

VLM test (not executed here)

```
# Run on a machine with a GPU:  pip install transformers torch timm einops
from transformers import AutoProcessor, AutoModelForCausalLM
from PIL import Image
import time
model_id = "microsoft/Florence-2-base"
model = AutoModelForCausalLM.from_pretrained(model_id,
trust_remote_code=True).eval()
proc = AutoProcessor.from_pretrained(model_id, trust_remote_code=True)
PROMPT = "<MORE_DETAILED_CAPTION>"     # then look for words such as spot,
bruise, black, damaged
for path in mango_paths[:10]:
    img = Image.open(path).convert("RGB")
    t = time.perf_counter()
    inp = proc(text=PROMPT, images=img, return_tensors="pt")
    out = model.generate(**inp, max_new_tokens=60, do_sample=False)   # greedy =
repeatable
    text = proc.batch_decode(out, skip_special_tokens=True)[0]
    print(path, round((time.perf_counter() - t) * 1000), "ms", text)
```

Final Decision

I choose **Approach C (automated computer vision)** as the main grader, with a human audit. **Not A:** it was the least consistent (kappa 0.64 between graders, 0.59 for the same grader), accuracy fell with fatigue (87.8 % → 81.1 %), and 8 fruits/s needs about 14–18

graders. **Not B:** the aid raised accuracy to 88.9 %, but a human still decides every fruit, so speed and fatigue remain. **C:** it is the only approach that meets the throughput (12.6 fruits/s on one core), gives the same answer every time and logs every fruit.

Limits, stated plainly: C was *less accurate* than the aided graders on my 30 fruits (76.7 %), because it misses faint bruises and over-rejects some fruit with small spots. So I would keep a 1 % human audit and a "send to review" bin, and I would improve the bruise detector (Tasks 7 and 10). The human results rest on simulated graders, so this comparison must be repeated with real graders.

Evidence provided: grading sheet (Table 2), feature comparison (Table 3), agreement and timing with Cohen's kappa (Table 4), the four related fields (Table 5), the capacity calculation and a justified choice with its limits.

TASK 2 – Levels of Computer Vision and Image Processing: Compare Pipeline Depths [5 Marks]

A vision system can stop at different levels: low-level (image in → image out), mid-level (image in → attributes out) and high-level (attributes in → understanding/decision out). Compare three pipeline depths for grading.

Technique	Basic idea and key concepts / functions
A. Low-level only	Pixel operations only – cv2.GaussianBlur, cv2.cvtColor, cv2.threshold – and a fixed rule on the count of dark pixels decides "defective".
B. Low + mid-level	Adds segmentation and feature extraction – cv2.findContours, cv2.contourArea, cv2.minAreaRect, mean hue, number of spots – and a rule-based grade.
C. Low + mid + high-level	Adds interpretation: features fed to a simple classifier (cv2.ml.KNearest or cv2.ml.SVM) that outputs the export grade with a confidence.

A computer vision pipeline works at three levels. The **low level** takes pixels and gives pixels (filtering, colour conversion, thresholding). The **mid level** takes pixels and gives attributes of regions (contours, length, hue, number and size of spots). The **high level** takes attributes and gives a meaning or a decision (export or reject). I built three techniques that use more and more levels and tested them on the same images: **A** counts dark pixels inside the fruit (low level only); **B** measures attributes and applies the buyer's rules (low + mid); **C** gives the same attributes to a 5-nearest-neighbour classifier (low + mid + high, learned).

Q1. *Simulate three computer vision techniques for fruit grading using Python/OpenCV and self-collected images. Submit commented source code and obtain all experimental results through code execution or actual measurements.*

Ans:

Data. Training set: 120 mangoes (seed 3). Test set: 60 ordinary mangoes (seed 4) plus 18 hard cases: 6 touching fruit, 6 leaf occlusions and 6 shadow bands. One third of the hard cases carry five spots, so they are defective. I added the hard cases on purpose, because a pipeline that works only on ideal images is not useful in a packing shed.

Fair comparison. Technique A has one free parameter, the dark-pixel threshold. I did not guess it; I chose the value that gave the highest accuracy on the training set (42 pixels). Technique C is trained on the same training set, and Technique B uses the buyer's rules directly. The feature vector of B and C has six values: number of spots, largest spot diameter, streak present, bruise area, mean hue and length.

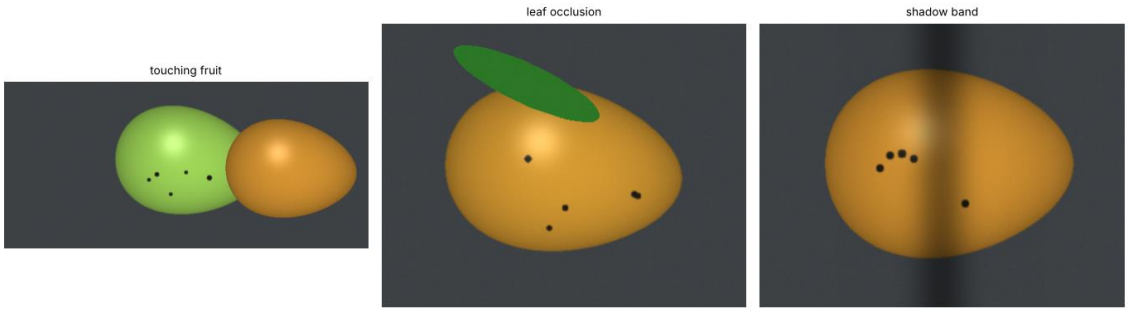


Figure 2: The three hard cases added to the test set

t2.py (Task 2 experiment: three pipeline depths, hard cases, pipeline figure)

```
from common import *
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.patches as mp
def hard(rng, kind, defective):
    k = 'manyspots' if defective else 'clean'
    if kind == 'touching':
        cw = (230, 105); a = dict(cls=mango.CLASSES[int(rng.integers(3))],
length_mm=float(rng.uniform(80, 100)), canvas_mm=cw)
        if defective: a.update(n_spots=5, spot_mm=(2.2, 3.2))
        im1, g1 = mango.render(rng, **a); im2, g2 = mango.render(rng,
cls='ripe', length_mm=float(rng.uniform(80, 100)), canvas_mm=cw)
        dx = int(0.30 * im1.shape[1]); m2 = np.roll(g2['mask'], dx, 1); i2 =
np.roll(im2, dx, 1)
        out = np.where(m2[..., None], i2, im1); gt = g1; gt['kind'] =
'touching'; return out, gt
    a = dict(cls=mango.CLASSES[int(rng.integers(3))],
length_mm=float(rng.uniform(80, 100)))
    if defective: a.update(n_spots=5, spot_mm=(2.2, 3.2))
    im, gt = mango.render(rng, **a); H, W = im.shape[:2]
    if kind == 'leaf':
        leaf = np.zeros((H, W), np.uint8)
        cx, cy = gt['center']; pts = cv2.ellipse2Poly((int(cx - 0.10 * W),
int(cy - 0.30 * H)), (int(0.22 * W), int(0.07 * H)), 25, 0, 360, 10)
        cv2.fillPoly(leaf, [pts], 255); lc = mango.hsv2bgr(58, 170, 120)
        im = np.where(leaf[..., None] > 0, (lc + rng.normal(0, 2,
im.shape)).astype(np.uint8), im)
    if kind == 'shadow':
        yy, xx = np.mgrid[0:H, 0:W].astype(np.float32); band = np.exp(-((xx - W
* rng.uniform(0.40, 0.6)) ** 2) / (2 * (0.05 * W) ** 2))
        im = np.clip(im * (1 - 0.45 * band[..., None]), 0, 255).astype(np.uint8)
    gt['kind'] = kind; return im, gt
def dark_count(bgr):
    hsv = cv2.cvtColor(cv2.GaussianBlur(bgr, (5, 5), 0), cv2.COLOR_BGR2HSV)
    f = ((hsv[..., 1] > 70) & (hsv[..., 2] > 50)).astype(np.uint8)
    f = cv2.morphologyEx(f, cv2.MORPH_CLOSE, np.ones((9, 9), np.uint8)) > 0
    return int(((hsv[..., 2] < 70) & f).sum())
def feats(bgr, mmpx):
    f = mango.measure(bgr, mmpx)
    if f is None: return np.zeros(6, np.float32)
    return np.array([len(f['spots']), max(f['spots'] + [0]), float(f['streak']),
f['bruise_area'], f['hue'], f['length']], np.float32)
train = mango.dataset(120, 3); test = mango.dataset(60, 4)
rng = np.random.default_rng(21)
hc = [hard(rng, k, bool(i % 3 == 0)) for i, k in enumerate(['touching'] * 6 +
['leaf'] * 6 + ['shadow'] * 6)]
test = test + hc
```

```
y_tr = np.array([int(mango.is_defective(g)) for _, g in train]); y_te =
np.array([int(mango.is_defective(g)) for _, g in test])
kinds = np.array([g['kind'] for _, g in test])
Xtr = np.array([feats(im, g['mm_per_px']) for im, g in train]); mu, sd =
Xtr.mean(0), Xtr.std(0) + 1e-6
knn = KNeighborsClassifier(5).fit((Xtr - mu) / sd, y_tr)
cnt = np.array([dark_count(im) for im, _ in train]); cands = np.unique(cnt)
thr = int(cands[int(np.argmax([((cnt > c) == y_tr).mean() for c in cands]))])
res = {'thrA': thr}
for name in 'ABC':
    pred, conf, ts = [], [], []
    for im, g in test:
        t = time.perf_counter()
        if name == 'A': p = int(dark_count(im) > thr)
        elif name == 'B':
            gr = mango.grade(im, g['mm_per_px']); p = int(gr['defective']) if gr
else 0
        else:
            x = (feats(im, g['mm_per_px']) - mu) / sd; pr =
knn.predict_proba(x[None])[0]; p = int(knn.classes_[pr.argmax()]);
conf.append(float(pr.max()))
        ts.append((time.perf_counter() - t) * 1000); pred.append(p)
    pred = np.array(pred); n_ord = 60
    res[name] = dict(acc_ord=float((pred[:n_ord] == y_te[:n_ord]).mean()),
acc_all=float((pred == y_te).mean()), ms=float(np.median(ts)),
        fp=int(((pred == 1) & (y_te == 0)).sum()), fn=int(((pred == 0) & (y_te
== 1)).sum()),
        by={k: float((pred[kinds == k] == y_te[kinds == k]).mean()) for k in
np.unique(kinds)}, conf=float(np.mean(conf)) if conf else None)
save('t2', res); print(res)
grid([hc[0][0], hc[6][0], hc[12][0]], ['touching fruit', 'leaf occlusion',
'shadow band'], f'{FIG}/t2_hard.png', size=4.2)
# pipeline diagram
fig, ax = plt.subplots(figsize=(11, 3.6)); ax.axis('off'); ax.set_xlim(0, 11);
ax.set_ylim(0, 4)
stages = [('Gaussian blur\ncv2.GaussianBlur', 'LOW', 'image → image'), ('BGR →
HSV\ncv2.cvtColor', 'LOW', 'image → image'), ('S,V threshold +
close\ncv2.threshold/morph', 'LOW', 'image → binary mask'),
          ('Contours, area, rect\ncv2.findContours', 'MID', 'mask → regions'),
('Length, hue, spots,\nbruise area', 'MID', 'regions → feature vector'), ('k-NN
(k=5)\nsklearn', 'HIGH', 'features → class + confidence'), ('Export / reject\n→
ejector', 'HIGH', 'label → action')]
col = {'LOW': '#cfe2f3', 'MID': '#d9ead3', 'HIGH': '#fce5cd'}
for i, (t, lv, io) in enumerate(stages):
    x = 0.15 + i * 1.55; ax.add_patch(mp.FancyBboxPatch((x, 1.6), 1.38, 1.5,
boxstyle='round,pad=0.03', fc=col[lv], ec='k'))
    ax.text(x + 0.69, 2.6, t, ha='center', va='center', fontsize=6.6); ax.text(x
+ 0.69, 1.85, lv, ha='center', fontsize=8, weight='bold')
    ax.text(x + 0.69, 1.25, io, ha='center', va='top', fontsize=6.2,
style='italic')
    if i < len(stages) - 1: ax.annotate('', xy=(x + 1.52, 2.35), xytext=(x +
1.4, 2.35), arrowprops=dict(arrowstyle='->'))
ax.set_title('Level-annotated pipeline of Technique C (low → mid → high)',
fontsize=10)
fig.savefig(f'{FIG}/t2_pipe.png', dpi=120, bbox_inches='tight'); plt.close(fig)
```

Q2. *Analyze the three techniques in terms of grading accuracy, handling of touching fruits and leaf occlusion, robustness to illumination variations, explainability of grading decisions, processing time per fruit, and effort required to incorporate a new grade.*

Ans:

Table 6: Accuracy per case type (same test set for all techniques)

Technique	clean	spots	many spots	streak	bruise	touching	leaf	shadow
A. Low only	100 %	50 %	100 %	70 %	0 %	100 %	100 %	33 %
B. Low + mid	85 %	90 %	100 %	100 %	50 %	33 %	33 %	33 %
C. Low + mid + high	50 %	90 %	100 %	100 %	30 %	100 %	100 %	83 %

Table 7: Qualitative comparison of the three techniques

Criterion	A. Low only	B. Low + mid	C. Low + mid + high
Touching fruit	only counts dark pixels, so the merged blob does not matter much	merged region gives wrong length and extra "spots"	same features as B, so same weakness
Leaf occlusion	leaf is not dark, little effect	green saturated leaf joins the fruit mask	same as B
Illumination / shadow	shadow pixels count as dark	shadow edge looks like a bruise	k-NN partly learned to ignore it
Explainability	one number	clear rules ("4 spots, largest 4.6 mm")	nearest examples + confidence
Effort to add a grade	retune one threshold	write and test new rules	add labelled examples and retrain

Technique C returns a confidence (the share of the five neighbours that agree); its mean was 0.82. This is useful for sending doubtful fruit to the review bin.

Q3. *Assess the performance of all three techniques using the same labelled image dataset, including challenging cases. Tabulate the measured grading accuracy and*

processing time in milliseconds per image, and determine whether each technique satisfies the decision-time limit of 1.2 seconds.

Ans:

Table 8: Accuracy and time on the same labelled test set

Technique	Acc. ordinary (60)	Acc. all (78)	False rejects	Misses	ms / image (median)	Within 1.2 s?
A. Low only	70.0 %	71.8 %	9	13	0.9	yes
B. Low + mid	85.0 %	73.1 %	16	5	78.1	yes
C. Low + mid + high	70.0 %	75.6 %	12	7	79.4	yes

All three are fast: the slowest median time is 79 ms per image, far inside 1.2 s. So speed does not decide the choice. Accuracy on ordinary fruit is 70.0 % for A, 85.0 % for B and 70.0 % for C.

Two results need a careful reading. First, **A looks good on touching fruit and leaves (100 %)** only because those cases contain no extra dark pixels and the threshold was tuned for them; it fails exactly where defects are subtle: bruises (0 %) and fruit with 1–3 spots (50 %). Second, **B is the most accurate on ordinary fruit but collapses on touching fruit and leaves** (33 % and 33 %), because the merged region gives a wrong outline and the stem end of the neighbour is counted as a spot. Technique C also has 100 % on these two, but its clean-fruit accuracy is only 50 %, so it rejects too many good fruit; with 78 test images this difference is only a few images and should not be over-interpreted.

Q4. *Design and illustrate the selected computer vision pipeline by identifying each OpenCV processing stage as low-level, mid-level, or high-level vision and specifying its input and output representations, such as images, regions, feature vectors, and class labels.*

Ans:

Figure 3 shows the selected pipeline. Each box gives the level, the OpenCV function and how the representation changes: image → mask → regions → feature vector → class → action.

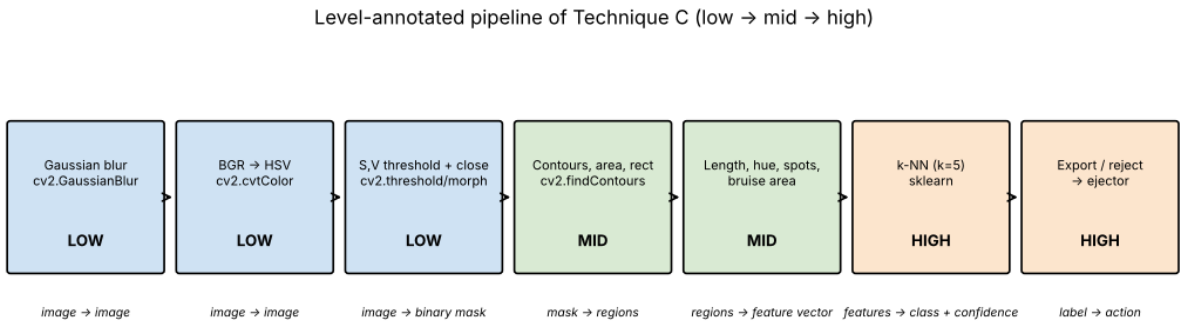


Figure 3: Level-annotated pipeline of Technique C

Q5. *Analyze the consequences of relying exclusively on low-level vision when dark stem ends or shadows are incorrectly identified as defects, and determine how inadequate mid-level features can affect the decisions made by the high-level grading stage.*

Ans:

Low level only. A low-level pipeline cannot tell a stem end, a shadow or a leaf vein from a defect, because they are all just "dark pixels". Technique A made 9 false rejects and 13 misses on the 78 test images. If I raise the threshold to stop the false rejects, small spots are missed immediately (only 50 % of the 1–3 spot fruit were right).

Poor mid-level features. The high level can only be as good as its features ("garbage in, garbage out"). When two touching fruit are merged into one region, the length and the spot count are wrong, and the decision stage is wrong even though the rule or the classifier is correct. For B the touching-fruit accuracy was 33 %. So the repair belongs at the mid level: split touching fruit with a distance transform and watershed, and remove leaves with a colour and shape test before measuring.

Q6. *Assess how end-to-end deep learning models, including Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs), learn low-level, mid-level, and high-level representations from labelled images. Compare their placement in the proposed pipeline and estimate their labelled-data requirements relative to the conventional computer vision approach.*

Ans:

I did not train a CNN or a ViT, so this part is reasoning. A CNN learns the three levels inside its layers. Early convolution filters respond to edges and colour (low level), middle layers respond to textures and parts such as spots and stem ends (mid level), and the last layers output the grade (high level). A Vision Transformer cuts the image into patches and uses attention between patches to group them, which plays the role of mid-level grouping. In my pipeline the network would replace the boxes from "threshold" to "k-NN" with one learned block; fruit segmentation, the trigger and the ejector would stay.

Labelled data. Training from scratch typically needs thousands of labelled images per class; fine-tuning a pretrained network needs a few hundred per class, and rare defects (stings, new bruise types) are the bottleneck. My classical pipeline needs almost no training labels, only a validation set of about 50–100 fruit to tune thresholds. The trade-off is accuracy on varied real fruit (deep learning) against explainability and low data needs (classical pipeline).

Final Decision

I choose **Technique C (low + mid + high level)**. **Not A:** low level only cannot separate defects from shadows or stem ends and misses bruises completely (0 %). **B or C:** both have a mid level and both run in under 80 ms. B was more accurate on ordinary fruit (85.0 % against 70.0 %), but C gives a confidence value for each decision, handled shadows better (83 % against 33 %), and can learn a new grade from labelled examples instead of new hand-written rules. **Limits:** on ordinary fruit C was less accurate than B, and neither handles touching fruit reliably in all cases, so mid-level segmentation must be improved or the fruit must be separated on the belt.

Evidence provided: all three techniques with code, per-case accuracy and timing (Tables 6–8), qualitative comparison (Table 7) and the level-annotated pipeline (Figure 3).

TASK 3 – Digital Image Fundamentals I: Compare Image Representations for Ripeness [4 Marks]

A digital image is a function f(x, y) stored as an M × N (× channels) array of numbers. Ripeness depends on peel colour, so the way colour is represented matters. Compare three representations.

Technique	Basic idea and key concepts / functions
A. RGB (BGR in OpenCV)	Thresholds on the three raw channels read by cv2.imread (BGR order).
B. HSV	cv2.cvtColor(img, cv2.COLOR_BGR2HSV); ripeness from hue H, with S and V used to ignore background and highlights.
C. CIE Lab*	cv2.cvtColor(img, cv2.COLOR_BGR2LAB); ripeness from a and b; colour difference ΔE between fruit and reference colours.

The grading line must sort mangoes into three ripeness classes (green, turning, ripe) by peel colour. The same colour can be written in different representations, and the choice matters because the shed light is not constant. I compared **A** RGB (stored as BGR by OpenCV), **B** HSV with hue H describing the kind of colour separately from saturation and brightness, and **C** CIE Lab with a* (green to red) and b* (blue to yellow).

Q1. *Apply (using Python/OpenCV): implement or simulate all three techniques on your own images and submit the commented code; every result below must come from running your own code or your own measurements.*

Ans:

Method. For each fruit I measured the mean peel colour in the three representations, ignoring the rim, the glossy highlight and, for hue, the red blush. I trained a nearest-centroid classifier (one average colour per class) with 45 fruit under **normal light only** and tested it on 45 fruit under normal, dim (0.6×) and bright (1.4×) light. This copies what happens in the shed: thresholds are set once and then the light changes.

t34.py, Task 3 part (colour features, lighting test, array demo)

```
# ---------------- Task 3 ----------------
def colour_feats(bgr, mmpx):
    mask, hsv = mango.segment(bgr)
    inner = cv2.erode(mask, np.ones((15, 15), np.uint8)) > 0
    V = hsv[..., 2]; inner &= V < np.percentile(V[inner], 95)
    b, g, r = [bgr[..., i][inner].mean() for i in range(3)]
    H = hsv[..., 0][inner]; keep = (H > 8) & (H < 160)
    h = mango.circ_mean_hue(H[keep]) if keep.any() else 0.0
    lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB)
```

```
    return dict(rgb=np.array([b, g, r]), hsv=np.array([h]),
lab=np.array([lab[..., 1][inner].mean() - 128, lab[..., 2][inner].mean() -
128]))
def centroid_clf(Xtr, ytr, X):
    cl = sorted(set(ytr)); cen = {c: np.mean([x for x, y in zip(Xtr, ytr) if y
== c], 0) for c in cl}
    return [min(cl, key=lambda c: np.linalg.norm(x - cen[c])) for x in X]
def make(n, light, seed):
    r = np.random.default_rng(seed)
    return [mango.render(r, cls=mango.CLASSES[i % 3], blush=(i % 3 == 2 and i %
2 == 0), light=light) for i in range(n)]
train = make(45, 1.0, 1); tests = {'normal (1.0x)': make(45, 1.0, 2), 'dim
(0.6x)': make(45, 0.6, 3), 'bright (1.4x)': make(45, 1.4, 4)}
Ftr = [colour_feats(im, g['mm_per_px']) for im, g in train]; ytr = [g['cls'] for
_, g in train]
acc = {}; feats = {}
for tn, items in tests.items():
    F = [colour_feats(im, g['mm_per_px']) for im, g in items]; y = [g['cls'] for
_, g in items]; feats[tn] = (F, y)
    for rep in ('rgb', 'hsv', 'lab'):
        p = centroid_clf([f[rep] for f in Ftr], ytr, [f[rep] for f in F]);
acc.setdefault(rep, {})[tn] = float(np.mean([a == b for a, b in zip(p, y)]))
sw = [colour_feats(im[..., ::-1].copy(), g['mm_per_px']) for im, g in
tests['normal (1.0x)']]
p = centroid_clf([f['hsv'] for f in Ftr], ytr, [f['hsv'] for f in sw]); swap_acc
= float(np.mean([a == b for a, b in zip(p, feats['normal (1.0x)'][1])]))
big = cv2.resize(train[0][0], (1280, 1024))
ms = dict(rgb=tms(lambda: big.copy(), 20), hsv=tms(lambda: cv2.cvtColor(big,
cv2.COLOR_BGR2HSV), 20), lab=tms(lambda: cv2.cvtColor(big, cv2.COLOR_BGR2LAB),
20))
bl = np.array([177, 178, 179, 0, 1, 2, 3], np.uint8)
img = np.zeros((1024, 1280, 3), np.uint8); a8 = np.full((1, 1), 250, np.uint8);
b8 = np.full((1, 1), 10, np.uint8)
save('t3', dict(acc=acc, swap_acc=swap_acc, ms=ms,
wrap=dict(arith=float(bl.mean()), circ=mango.circ_mean_hue(bl)), np_add=int((a8
+ b8)[0, 0]), cv_add=int(cv2.add(a8, b8)[0, 0]),
                store_bits=1024 * 1280 * 8 * 3, store_mb=1024 * 1280 * 3 / 1e6,
shape=list(img.shape)))
mk = {'green': 'o', 'turning': 's', 'ripe': '^'}; fig, axs = plt.subplots(1, 3,
figsize=(11, 3.6))
for tn, fill in [('normal (1.0x)', True), ('dim (0.6x)', False)]:
    F, y = feats[tn]
    for c in mango.CLASSES:
        idx = [i for i, v in enumerate(y) if v == c]
        kw = dict(marker=mk[c], s=22, edgecolors='k', facecolors='k' if fill
else 'none', lw=0.8, label=f'{c} ({tn.split()[0]})')
        axs[0].scatter([F[i]['rgb'][2] for i in idx], [F[i]['rgb'][1] for i in
idx], **kw)
        axs[1].scatter([F[i]['hsv'][0] for i in idx], [i for i in
range(len(idx))], **kw)
        axs[2].scatter([F[i]['lab'][0] for i in idx], [F[i]['lab'][1] for i in
idx], **kw)
axs[0].set(xlabel='mean R', ylabel='mean G', title='RGB: clusters slide with
light'); axs[1].set(xlabel='hue H (0-180)', ylabel='fruit index', title='HSV:
hue stays in place'); axs[2].set(xlabel='a*', ylabel='b*', title='Lab (a*, b*)')
axs[1].legend(fontsize=5, ncol=2); fig.tight_layout();
fig.savefig(f'{FIG}/t3_scatter.png', dpi=110); plt.close(fig)
print('T3', load()['t3'])
```

Q2. *Compare on: separation of green / turning / ripe classes, sensitivity to brightness change, effect of red blush, behaviour of hue near 0°/180° wrap-around, and computation time.*

Ans:

Table 9: Ripeness accuracy (45 fruit per lighting level, nearest-centroid classifier)

Representation	Normal	Dim (0.6×)	Bright (1.4×)	Convert time, 1280×1024
A. RGB (BGR)	100.0 %	66.7 %	55.6 %	0.4 ms
B. HSV (hue)	100.0 %	100.0 %	100.0 %	0.8 ms
C. Lab (a*, b*)	100.0 %	100.0 %	100.0 %	3.4 ms

- **Brightness change:** in RGB the class clusters slide towards the origin when the light is dimmed, so fixed thresholds fail: accuracy fell from 100.0 % to 66.7 % in dim light and 55.6 % in bright light. Hue is a ratio of the channels, so it stays in place: HSV was 100.0 % correct at all three light levels. Lab also stayed at 100 % in my test, because a* and b* are largely independent of lightness L*.
- **Red blush:** blush pixels have a hue near 0 and 180 (OpenCV hue runs from 0 to 180). The ordinary average of the sample values 177, 178, 179, 0, 1, 2, 3 is **77.1**, which would look green, but the circular mean is **0.0**. So the grader averages hue as an angle and excludes blush from the ripeness hue.
- **Computation time:** HSV conversion took 0.8 ms and Lab 3.4 ms per 1.3 MP frame; both are small compared with the 125 ms available per fruit, but Lab costs about 4.1 times as much as HSV.

Q3. *Justify with evidence: plot the class distributions (histograms or scatter plots) of the chosen channels for at least 30 fruits under two lighting levels, and report classification accuracy for each representation.*

Ans:

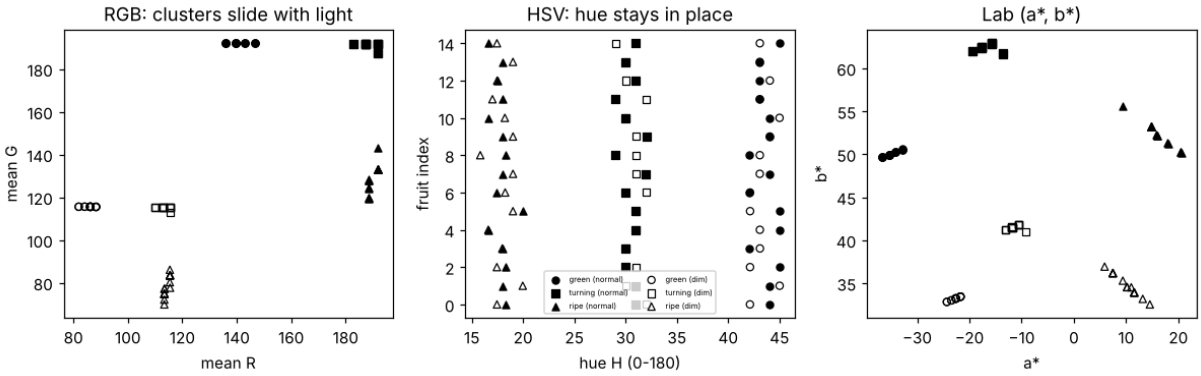


Figure 4: Class distributions in the three representations (filled = normal light, open = dim light)

Figure 4 shows why RGB fails: the open markers (dim light) are shifted away from the filled ones in the R–G plane, while the hue values of the same fruit do not move. The accuracy table above reports the result for all three representations.

Q4. *Show the array view of your image: shape, dtype, the (row, column) vs (x, y) convention in NumPy, the storage formula b = M × N × k bits, and the uint8 overflow difference between cv2.add and NumPy "+".*

Ans:

In Python an image is a NumPy array of M rows × N columns × channels, with k bits per value. The program output is:

Output of the array demonstration

```
img.shape = (1024, 1280, 3)    (rows M, columns N, channels)
img.dtype = uint8   -> k = 8 bits per channel
pixel at x=100 (column), y=20 (row) is img[20, 100]    (row first in NumPy)
b = M x N x k x channels = 1024 x 1280 x 8 x 3 = 31,457,280 bits = 3.93 MB
NumPy  a + b    = 4     (250 + 10 wraps around: 260 mod 256)
OpenCV cv2.add = 255  (saturates at 255)
```

- **Row first:** the pixel at x = 100 (column) and y = 20 (row) is img[20, 100]. Writing img[100, 20] is a common mistake.
- **Storage:** b = M × N × k × channels = 1024 × 1280 × 8 × 3 = 31,457,280 bits = 3.93 MB for one uncompressed colour frame.
- **Overflow:** NumPy uint8 addition wraps around (250 + 10 = 4), but cv2.add saturates at 255. A brightness correction written with "+" in NumPy can therefore turn a bright peel pixel almost black.

Q5. *What if not: What if ripeness is graded in RGB and the roof daylight doubles? What if the image is accidentally treated as RGB instead of BGR?*

Ans:

RGB when the daylight doubles. Every channel roughly doubles and saturates at 255, so a green fruit can look yellow. In my test RGB accuracy was already 55.6 % at only 1.4× light, so at 2× it would be worse.

Image treated as RGB instead of BGR. cv2.imread returns BGR. If the channels are read as RGB, red and blue swap, the hue moves to the wrong place and the accuracy of the HSV classifier fell to **33.3 %**, which is chance level for three classes. So images read with cv2.imread must always be converted with COLOR_BGR2HSV or COLOR_BGR2LAB.

Q6. *Compare your chosen colour space with features from a pretrained network (for example, a ResNet or DINOv2 embedding) for ripeness on the same 30 fruits, and judge which is easier to explain to the buyer.*

Ans:

I did not run a ResNet or DINOv2 embedding (no model download), so this is reasoning. A pretrained network turns each image into an embedding of 512–768 numbers. I expect such an embedding to separate the three ripeness classes well and to be fairly robust to lighting, probably comparable with HSV. But it is hard to explain to a buyer ("dimension 341 is high"), whereas "mean hue 19 = ripe" can be checked with a colour chart. I would keep hue for the grading rule and consider embeddings only for a quality attribute that cannot be described by colour. The code to compare them is below.

Embedding experiment (not executed here)

```
# with torch + timm installed (GPU optional)
import timm, torch
m = timm.create_model("resnet18", pretrained=True, num_classes=0).eval()
cfg = timm.data.resolve_data_config({}, model=m); tf =
timm.data.create_transform(**cfg)
emb = torch.stack([m(tf(img)[None])[0] for img in pil_images]).detach().numpy()
# then nearest-centroid / k-NN on emb, exactly as for the colour features
```

Final Decision

I choose **B – HSV (hue)**. **Not A (RGB):** it changes with the amount of light (100.0 % → 55.6 %). **Lab:** it was as accurate as HSV in my test, and it is the right choice if the buyer asks for a colour difference ΔE, but it costs about 4.1 times the conversion time. **B:** 100.0 % correct under all three light levels, the cheapest of the three to compute after RGB, and easy to explain. Hue must be averaged as an angle and blush excluded. **Limit:** my images are synthetic and the three classes are well separated, so on real fruit with variety differences the margin will be smaller.

Evidence provided: code, accuracy under three lighting levels with conversion times (Table 9), class-distribution plots (Figure 4), array view and storage calculation, and the what-if analysis.

TASK 4 – Digital Image Fundamentals II: Compare Pixel Connectivity for Defect Counting [4 Marks]

The buyer rejects a fruit with more than 3 black spots or any spot larger than 4 mm. After thresholding, how pixels are grouped into "spots" decides the count. Compare three connectivity rules.

Technique	Basic idea and key concepts / functions
A. 4-connectivity	Pixels joined only through N4 neighbours; cv2.connectedComponentsWithStats(binary, connectivity=4).
B. 8-connectivity	Pixels joined through N8 neighbours (including diagonals); cv2.connectedComponentsWithStats(binary, connectivity=8).
C. m-connectivity (mixed)	Diagonal links allowed only when no shared 4-neighbour exists; implement your own labelling in Python (not built into OpenCV).

The buyer's rule depends on **counting** spots (more than 3 means reject) and **measuring** them (any spot over 4 mm means reject). After thresholding, each spot is a group of dark pixels, and the connectivity rule decides which pixels belong to the same spot. **4-connectivity** joins a pixel only to its left, right, upper and lower neighbours. **8-connectivity** also joins the four diagonal neighbours. **m-connectivity** (mixed) joins 4-neighbours always, and a diagonal neighbour only if the two pixels they share as 4-neighbours are both background; this keeps the diagonal links of 8-connectivity but removes double paths.

Q1. *Develop three connected-component labelling techniques using Python/OpenCV to identify and count fruit defects. Submit commented source code and generate experimental results through code execution or actual measurements.*

Ans:

OpenCV provides 4- and 8-connected labelling (cv2.connectedComponents), but not m-connectivity, so I wrote my own two-pass labelling with union-find. In the first pass each foreground pixel looks at its already visited neighbours: with none it gets a new label, otherwise it takes the smallest label and the labels are recorded as equivalent. In the second pass every label is replaced by its root. I also wrote a function that counts adjacency links (graph edges), because the number of components alone cannot show the difference between 8- and m-connectivity.

t34.py, Task 4 part (own m-connectivity labelling, link counting, tests)

```
# ---------------- Task 4 ----------------
```

```
def label_m(binary):
    b = binary.astype(bool); H, W = b.shape; lab = np.zeros((H, W), np.int32);
parent = [0]
    def find(x):
        while parent[x] != x: parent[x] = parent[parent[x]]; x = parent[x]
        return x
    def union(x, y):
        rx, ry = find(x), find(y)
        if rx != ry: parent[max(rx, ry)] = min(rx, ry)
    for y in range(H):
        for x in range(W):
            if not b[y, x]: continue
            nb = []
            if x > 0 and b[y, x - 1]: nb.append(lab[y, x - 1])
            if y > 0 and b[y - 1, x]: nb.append(lab[y - 1, x])
            if y > 0 and x > 0 and b[y - 1, x - 1] and not b[y - 1, x] and not
b[y, x - 1]: nb.append(lab[y - 1, x - 1])
            if y > 0 and x < W - 1 and b[y - 1, x + 1] and not b[y - 1, x] and
not b[y, x + 1]: nb.append(lab[y - 1, x + 1])
            if not nb: parent.append(len(parent)); lab[y, x] = len(parent) - 1
            else:
                m = min(find(n) for n in nb); lab[y, x] = m
                for n in nb: union(m, n)
    roots = {}
    for y, x in zip(*np.nonzero(lab)): lab[y, x] = roots.setdefault(find(lab[y,
x]), len(roots) + 1)
    return len(roots), lab
def links(binary, mode):
    b = binary.astype(bool); H, W = b.shape; n = 0
    for y in range(H):
        for x in range(W):
            if not b[y, x]: continue
            for dy, dx in [(0, 1), (1, 0)]: n += int(y + dy < H and x + dx < W
and b[y + dy, x + dx])
            for dy, dx in [(1, 1), (1, -1)]:
                yy, xx = y + dy, x + dx
                if 0 <= yy < H and 0 <= xx < W and b[yy, xx] and (mode == '8' or
(not b[y, xx] and not b[yy, x])): n += 1
    return n
def pats():
    P = {}
    a = np.zeros((20, 20), np.uint8); a[4:8, 4:8] = 1; a[8:12, 8:12] = 1; P['two
spots touching at a corner'] = (a, 2)
    a = np.zeros((20, 20), np.uint8)
    for i in range(3, 16): a[i, i] = 1
    P['1-px diagonal streak'] = (a, 1)
    a = np.zeros((20, 20), np.uint8); a[2:5, 2:5] = 1; a[8:11, 9:12] = 1;
a[14:17, 3:6] = 1; P['three separate spots'] = (a, 3)
    a = np.zeros((20, 20), np.uint8); a[5:10, 4:8] = 1; a[5:10, 8:12] = 1;
P['two spots merged side by side'] = (a, 1)
    a = np.zeros((20, 20), np.uint8); a[3:12, 3] = 1; a[11, 3:10] = 1; a[3:5, 4]
= 1; a[16, 16] = 1; a[2, 17] = 1; P['L-shape + 2 noise pixels'] = (a, 3)
    return P
rows9 = []
for name, (a, true) in pats().items():
    c4 = cv2.connectedComponents(a, connectivity=4)[0] - 1; c8 =
cv2.connectedComponents(a, connectivity=8)[0] - 1; cm = label_m(a)[0]
    rows9.append([name, true, c4, c8, cm, links(a, '8'), links(a, 'm')])
rng = np.random.default_rng(41); rows10 = []; t4 = []; t8 = []; tm = []
for i in range(10):
```

```
    im, gt = mango.make(rng, ['spots', 'manyspots', 'streak', 'spots',
'manyspots'][i % 5], cls='ripe', blush=False)
    mask, hsv = mango.segment(im); inner = cv2.erode(mask, np.ones((15, 15),
np.uint8)) > 0
    V = hsv[..., 2].astype(np.float32); bw = ((V < 0.55 * np.median(V[inner])) &
inner).astype(np.uint8)
    bw[rng.random(bw.shape) < 0.0004] = 1; bw = bw &
(cv2.dilate(inner.astype(np.uint8), np.ones((3, 3))) > 0)
    true = len(gt['spots']) + (1 if gt['streak'] else 0)
    def cc(conn, amin):
        n, lab, st, _ = cv2.connectedComponentsWithStats(bw, connectivity=conn);
return int(sum(st[j, cv2.CC_STAT_AREA] >= amin for j in range(1, n)))
    mm = label_m(bw)[1]; cm_raw = int(mm.max()); cm_f = int(sum((mm == j).sum()
>= 3 for j in range(1, mm.max() + 1)))
    rows10.append([i + 1, true, cc(4, 1), cc(8, 1), cm_raw, cc(4, 3), cc(8, 3),
cm_f])
    if i == 0:
        t4.append(tms(lambda: cv2.connectedComponents(bw, connectivity=4), 20));
t8.append(tms(lambda: cv2.connectedComponents(bw, connectivity=8), 20));
tm.append(tms(lambda: label_m(bw), 2))
# distance measures for a spot drawn at 45 deg (ellipse long axis along
diagonal)
rows11 = []
for d in (2.0, 3.0, 4.0, 4.5, 5.0):
    mp_ = 0.25; img = np.zeros((120, 120), np.uint8); L = int(round(d /
(np.sqrt(2) * mp_))); cv2.line(img, (60 - L // 2, 60 - L // 2), (60 + L // 2, 60
+ L // 2), 1, 1)
    ys, xs = np.nonzero(img); dx = xs.max() - xs.min() + 1; dy = ys.max() -
ys.min() + 1
    rows11.append([d, float(np.hypot(dx, dy) * mp_), float((dx + dy) * mp_),
float(max(dx, dy) * mp_)])
save('t4', dict(rows9=rows9, rows10=rows10, t4=t4[0], t8=t8[0], tm=tm[0],
rows11=rows11)); print('T4', rows9, rows10, rows11, t4, t8, tm)
# figure: streak under 4 vs 8
a = pats()['1-px diagonal streak'][0]; n4, l4 = cv2.connectedComponents(a,
connectivity=4); n8, l8 = cv2.connectedComponents(a, connectivity=8)
fig, ax = plt.subplots(1, 3, figsize=(10, 3.4)); ax[0].imshow(a, cmap='gray');
ax[0].set_title('binary streak'); ax[1].imshow(l4, cmap='tab20');
ax[1].set_title(f'4-conn: {n4-1} labels'); ax[2].imshow(l8, cmap='tab20');
ax[2].set_title(f'8-conn: {n8-1} label')
for x in ax: x.axis('off')
fig.tight_layout(); fig.savefig(f'{FIG}/t4_streak.png', dpi=110); plt.close(fig)
```

Q2. *Examine the behaviour of the three techniques with respect to touching and diagonally adjacent spots, double-path ambiguity, noise-pixel sensitivity, variations in measured spot size, and execution time.*

Ans:

- **Touching and diagonal spots:** two spots meeting only at a corner are 2 spots for 4-connectivity but 1 for 8- and m-connectivity. Two spots overlapping side by side are 1 spot for all three rules, so no connectivity rule can split them; that needs shape analysis such as a distance transform with watershed.

- **Diagonal streaks:** a 1-pixel diagonal line is broken into many pieces by 4-connectivity (Figure 5 and Table 10). A sap-burn streak would then be counted as many spots instead of one streak.
- **Double-path ambiguity:** for the same pixels, 8-adjacency counts redundant links (a pixel pair joined both directly and through a common neighbour); m-adjacency removes them. The number of components is the same, so m-connectivity matters for tracing paths and boundaries, not for counting.
- **Noise pixels:** single noise pixels become extra "spots" for every rule. A minimum area of 3 pixels removes them.
- **Execution time:** 4-connectivity 0.10 ms and 8-connectivity 0.07 ms (OpenCV, C++) against 9.6 ms for my pure-Python m-labelling, on the same 432 × 336 image.

Q3. *Validate the defect-counting performance using at least five synthetic binary patterns and ten thresholded fruit images. Present a tabular comparison of automatically detected spot counts and manually verified reference counts, supported by experimental evidence.*

Ans:

Table 10: Counts on the five synthetic patterns (true = known count)

Pattern	True	4-conn	8-conn	m-conn	Links (8)	Links (m)
two spots touching at a corner	2	2	1	1	85	49
1-px diagonal streak	1	13	1	1	12	12
three separate spots	3	3	3	3	60	36
two spots merged side by side	1	1	1	1	123	67
L-shape + 2 noise pixels	3	3	3	3	21	17

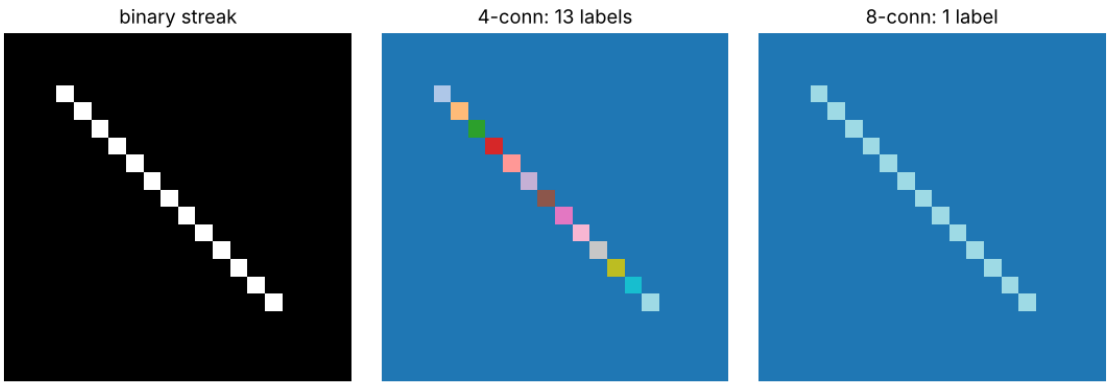


Figure 5: A 1-pixel diagonal streak – 4-connectivity splits it into many labels, 8-connectivity keeps it whole

Table 11: Counts on 10 thresholded fruit images (0.3125 mm/px, with random noise pixels)

Image	True	4-conn	8-conn	m-conn	4 + area ≥ 3	8 + area ≥ 3	m + area ≥ 3
1	3	20	20	20	3	3	3
2	7	25	25	25	7	7	7
3	1	31	31	31	1	1	1
4	1	14	14	14	1	1	1
5	5	10	10	10	5	5	5
6	3	17	17	17	3	3	3
7	5	24	24	24	5	5	5
8	1	24	24	24	1	1	1
9	2	12	12	12	2	2	2
10	5	26	26	26	5	5	5

Without a minimum area, every rule counts noise pixels as spots (for example image 1 gives 20 labels for 3 true spots). After the 3-pixel area filter the counts matched the true count in all 10 images for all three rules. This is an honest limit of my test: the streaks in my fruit images are 1.5 mm wide (about 5 pixels), so even 4-connectivity does not break them. The splitting problem is seen only on the 1-pixel diagonal line of the synthetic pattern, which is what a very thin sap-burn trace or an under-sampled streak would look like.

Q4. *Calculate defect diameters and inter-spot gaps using Euclidean (De), city-block (D4), and chessboard (D8) distance measures. Convert pixel measurements into millimetres using image calibration and establish which measurement method satisfies the buyer's 4 mm criterion.*

Ans:

To check the 4 mm rule I need the size of each spot. I compared three distance measures on a spot drawn along a 45° diagonal, where they differ most: Euclidean **$De = \sqrt{(dx^2 + dy^2)}$**, city-block **$D4 = |dx| + |dy|$** and chessboard **$D8 = \max(|dx|, |dy|)$**, converted to millimetres with mm = pixels × mm-per-pixel (0.25 mm/px).

Table 12: Measured largest extent of a spot drawn at 45° (0.25 mm/px)

True size (mm)	De (Euclidean)	D4 (city-block)	D8 (chessboard)
2.0	2.47	3.50	1.75
3.0	3.18	4.50	2.25
4.0	3.89	5.50	2.75
4.5	4.60	6.50	3.25
5.0	5.30	7.50	3.75

D4 over-estimates a diagonal size by up to a factor of √2 and would reject a 3 mm spot as "over 4 mm" (4.50 mm). D8 under-estimates it and would accept a 5 mm spot (3.75 mm). Only De is the same in every direction (here 4.60 mm for a true 4.5 mm spot, within one pixel), so **De converted with the calibrated mm-per-pixel is the measure that satisfies the 4 mm criterion.**

Q5. *Investigate the effects of 4-connectivity splitting a diagonal sap-burn streak into multiple components and 8-connectivity combining separate defects into a single component. Formulate an appropriate connectivity-selection strategy for reliable fruit inspection.*

Ans:

4-connectivity splitting a diagonal streak: one streak becomes many small components; each is below 4 mm, so the streak rule is never triggered and the count rule may fire wrongly. **8-connectivity merging separate defects:** two spots that touch at a corner become one component, so the count is too low and the merged object can look larger than 4 mm (a false reject). My selection strategy is therefore:

- threshold → small closing → 8-connected labelling → remove components smaller than 3 pixels;
- classify each component by shape: elongated (length / width > 3) means streak, otherwise spot;
- measure each spot with De (equivalent-circle diameter or largest Euclidean extent);
- if a component is unusually large or not convex, split it with a distance-transform watershed before counting.

Q6. *Recommend a suitable defect-segmentation approach by comparing connectivity-based labelling with instance-segmentation models, such as YOLO-seg or SAM 2, using the same test images. Support the recommendation with evidence on individual defect*

masks, spot counts, touching and diagonal defects, measurement accuracy, and computational cost.

Ans:

I did not run YOLO-seg or SAM 2 (no GPU, no model download), so this is reasoning and not a measurement. Instance-segmentation models output one mask per defect, so they can separate touching spots that no connectivity rule can split, and SAM 2 can segment a defect from a single click. But YOLO-seg needs a few hundred labelled defect masks to learn 2 mm spots, both models need a GPU for real-time use, and SAM 2 is far too slow for 8 fruits/s on a CPU. The size measurement would still be De on each mask. So I would keep connectivity-based labelling on the line (fast and explainable) and use SAM 2 offline to speed up the labelling of new defect images.

Final Decision

I choose **8-connectivity with a 3-pixel minimum-area filter for counting, and the Euclidean distance De for spot size. Not 4-connectivity:** it breaks thin diagonal streaks into many pieces (13 labels for one 1-pixel streak). **Not m-connectivity for counting:** it always gave the same number of components as 8-connectivity; it only removes redundant diagonal links, which matters for tracing boundaries, and my Python version took 9.6 ms against 0.07 ms for OpenCV. **8-connectivity + area filter + De:** keeps streaks whole, removes noise pixels, and applies the 4 mm rule in the same way in every direction. **Limit:** touching spots that overlap cannot be separated by any connectivity rule and need watershed or an instance-segmentation model.

Evidence provided: own m-connectivity code, counts on synthetic patterns and fruit images (Tables 10 and 11), distance-measure comparison (Table 12), the streak figure and the selection strategy.

TASK 5 – Image Sensing and Acquisition: Compare Sensor Arrangements [5 Marks]

The fruit moves at 0.5 m/s under the camera. How the scene energy is sensed decides whether the image is sharp, undistorted and correctly coloured. Compare three acquisition arrangements.

Technique	Basic idea and key concepts / functions
A. Area-scan rolling-shutter CMOS (USB)	Sensor array with rows exposed one after another; continuous lighting; cv2.VideoCapture with manual exposure (CAP_PROP_EXPOSURE).
B. Area-scan global-shutter CMOS with strobe	Sensor array with all pixels exposed together; short LED flash triggered by a belt encoder.
C. Line-scan camera (sensor strip)	A single row of sensors; the belt motion builds the 2-D image line by line, synchronized to an encoder.

The mangoes move at 0.5 m/s, so the camera must capture a moving object without blurring the smallest defects. I compared **A** an area-scan rolling-shutter USB camera (exposure 1–4 ms, room light), **B** an area-scan global-shutter camera with a 0.1 ms LED strobe, and **C** a line-scan camera that builds the image one row at a time.

Q1. *Demonstrate three imaging arrangements using Python/OpenCV: (A) area-scan imaging with motion blur, (B) rolling-shutter imaging with row-wise displacement, and (C) line-scan imaging through sequential line acquisition and stacking. Where feasible, capture images of a moving object and submit commented source code with experimentally obtained results.*

Ans:

I simulated each arrangement on the same generated fruit images, so the only difference is how the image is captured. I could not capture a real moving object (no camera and belt), so these are simulations of the physics, and I say so wherever it matters.

- **Area-scan exposure blur:** while the shutter is open the fruit moves, so each point is smeared along the belt by v × t. I model this by averaging the image along the belt direction over that length.
- **Rolling shutter:** row y starts y × line-time later than row 0, by when the fruit has moved (skew). A 50 Hz tube light flickers at 100 Hz, so each row sees a different light level, giving bright and dark bands.
- **Line-scan:** one row per belt step. If the line rate is synchronised with a belt encoder the image has the correct shape. If not, a belt that runs 10 % fast compresses the fruit along the belt.

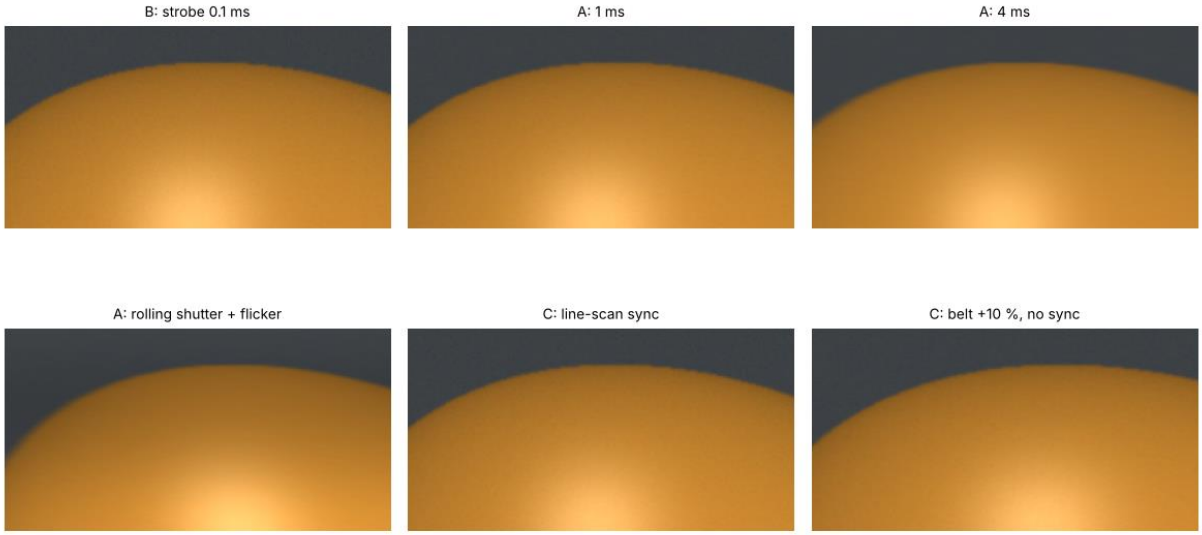


Figure 6: The same fruit as seen by each arrangement

acq.py (motion blur, rolling shutter, line-scan, JPEG helpers)

```
from common import *
V_BELT = 500.0   # mm/s
def motion_blur(img, blur_mm, mmpx):
    L = blur_mm / mmpx
    if L < 1: return img
    n = int(np.ceil(L)); k = np.zeros((1, n + 1), np.float32); k[0, :n] = 1; k
/= k.sum()
    return cv2.filter2D(img, -1, k, borderType=cv2.BORDER_REFLECT)
def rolling(img, mmpx, readout_ms=30.0, exposure_ms=4.0, flicker=True,
v=V_BELT):
    H = img.shape[0]; lt = readout_ms / H; out = np.empty(img.shape, np.float32)
    for y in range(H):
        sh = v * (y * lt) / 1000 / mmpx
        out[y] = cv2.warpAffine(img[y:y + 1], np.float32([[1, 0, sh], [0, 1,
0]]), (img.shape[1], 1), borderMode=cv2.BORDER_REPLICATE)[0]
    out = motion_blur(out, v * exposure_ms / 1000, mmpx).astype(np.float32)
    if flicker:
        t = np.arange(H) * lt / 1000; damp = abs(np.sinc(100 * exposure_ms /
1000)); gain = 1 + 0.12 * damp * np.sin(2 * np.pi * 100 * t); out = out *
gain[:, None, None]
    return np.clip(out, 0, 255).astype(np.uint8)
def line_scan(img, speed_factor=1.0):
    H, W = img.shape[:2]; return cv2.resize(img, (int(round(W / speed_factor)),
H), interpolation=cv2.INTER_AREA)
def jpeg(img, q): return cv2.imdecode(cv2.imencode('.jpg', img,
[cv2.IMWRITE_JPEG_QUALITY, q])[1], 1)
FAINT = (45, 65, 85)
```

t5.py (Task 5 experiment)

```
from acq import *
mmpx = 0.25
rng = np.random.default_rng(51); fr = []
```

```
for i in range(12):
    im, gt = mango.render(rng, cls='ripe', length_mm=float(rng.uniform(75,
105)), mm_per_px=mmpx, n_spots=3, spot_mm=(2.0, 2.0), spot_bgr=(22, 28, 30) if i
% 2 == 0 else FAINT)
    gt['dark'] = (i % 2 == 0); fr.append((im, gt))
cfgs = {'B: global shutter + 0.1 ms strobe': lambda im: motion_blur(im, V_BELT *
0.1 / 1000, mmpx),
        'A: area-scan 1 ms': lambda im: motion_blur(im, V_BELT * 1 / 1000,
mmpx),
        'A: area-scan 4 ms': lambda im: motion_blur(im, V_BELT * 4 / 1000,
mmpx),
        'A: rolling 30 ms + 4 ms + 100 Hz': lambda im: rolling(im, mmpx),
        'C: line-scan, synchronised': lambda im: im,
        'C: line-scan, belt +10 %, no sync': lambda im: line_scan(im, 1 / 1.10)}
rows = []; ex = []
for name, fn in cfgs.items():
    det = {True: 0, False: 0}; tot = {True: 0, False: 0}; lerr = []; sz = []
    for im, gt in fr:
        im2 = fn(im); sc = im2.shape[1] / im.shape[1]
        f = mango.measure(im2, mmpx, min_spot_mm2=0.3)
        n = min(len(f['spots']), 3) if f else 0; det[gt['dark']] += n;
tot[gt['dark']] += 3
        if f: lerr.append(abs(f['length'] - gt['length_mm'])); sz += f['spots']
    rows.append([name, det[True] / tot[True], det[False] / tot[False],
float(np.mean(sz)) if sz else 0.0, float(np.mean(lerr))]);
ex.append(fn(fr[1][0])[60:170, 120:330])
bay = {}
im, gt = fr[0]; flat = im.copy()
b, g, r = cv2.split(im); mos = np.zeros(im.shape[:2], np.uint8); mos[0::2, 0::2]
= b[0::2, 0::2]; mos[0::2, 1::2] = g[0::2, 1::2]; mos[1::2, 0::2] = g[1::2,
0::2]; mos[1::2, 1::2] = r[1::2, 1::2]
def _de(code):
    d = cv2.cvtColor(mos, code); return np.linalg.norm(cv2.cvtColor(im,
cv2.COLOR_BGR2LAB).astype(np.float32) - cv2.cvtColor(d,
cv2.COLOR_BGR2LAB).astype(np.float32), axis=2)[gt['mask']].mean()
code = min([cv2.COLOR_BAYER_BG2BGR, cv2.COLOR_BAYER_RG2BGR,
cv2.COLOR_BAYER_GB2BGR, cv2.COLOR_BAYER_GR2BGR], key=_de)   # pick the matching
pattern code
demo = cv2.cvtColor(mos, code)
lab0 = cv2.cvtColor(im, cv2.COLOR_BGR2LAB).astype(np.float32); lab1 =
cv2.cvtColor(demo, cv2.COLOR_BGR2LAB).astype(np.float32)
dE = np.linalg.norm(lab0 - lab1, axis=2); edge =
cv2.dilate(cv2.Canny(cv2.cvtColor(im, cv2.COLOR_BGR2GRAY), 40, 120), np.ones((3,
3))) > 0
flatm = gt['mask'] & ~edge & (cv2.erode(gt['mask'].astype(np.uint8),
np.ones((25, 25))) > 0)
calc = {f'{t} ms': dict(blur_mm=V_BELT * t / 1000, blur_px=V_BELT * t / 1000 /
mmpx) for t in (0.1, 1.0, 4.0)}
save('t5', dict(rows=rows, dE_edge=float(dE[edge & gt['mask']].mean()),
dE_flat=float(dE[flatm].mean()), calc=calc, line_rate=V_BELT / mmpx,
skew_mm=V_BELT * 0.030))
grid(ex, ['B: strobe 0.1 ms', 'A: 1 ms', 'A: 4 ms', 'A: rolling shutter +
flicker', 'C: line-scan sync', 'C: belt +10 %, no sync'], f'{FIG}/t5_arr.png',
cols=3, size=3.6)
print(load()['t5'])
```

Q2. *Examine the three imaging arrangements with respect to motion blur, calculated as belt speed multiplied by exposure time; rolling-shutter skew; sensitivity to 100 Hz*

illumination flicker; colour acquisition using a Bayer colour filter array versus a three-sensor camera; required frame or line rate; and implementation cost.

Ans:

- **Motion blur** = belt speed × exposure time (Table 13).
- **Rolling-shutter skew:** with a 30 ms readout the bottom row is exposed 30 ms after the top row, so the shift across the frame is 0.5 m/s × 30 ms = **15 mm**.
- **Flicker:** a tube light on 50 Hz mains flickers at 100 Hz (10 ms period). Each row of a rolling shutter samples a different phase of the cycle, so bands appear. A strobe much brighter than the room light removes the problem.
- **Colour:** a single-sensor camera with a Bayer filter measures one colour per pixel and interpolates the other two. I mosaicked and demosaicked a fruit image: the colour error was ΔE = **1.8** on flat peel but **4.8** on spot and fruit edges. That is acceptable for ripeness (an average over the whole peel) but slightly blurs spot edges. A three-sensor camera avoids this at several times the cost.
- **Required rates:** line-scan rate = belt speed / pixel size = 500 mm/s ÷ 0.25 mm = **2000 lines/s**. An area-scan camera needs one triggered frame per fruit (8 frames/s).
- **Cost:** A is cheapest; B needs a global-shutter camera, a strobe controller and a trigger; C needs an encoder, a long bright line light and a line-scan camera.

Q3. *Compute motion blur for exposure times of 0.1 ms, 1 ms, and 4 ms using the specified belt speed. Determine the required line-scan rate for Arrangement C and experimentally verify whether a 2 mm defect remains detectable under each simulated imaging condition.*

Ans:

Table 13: Motion blur at 0.5 m/s

Exposure	Blur = v × t	Blur in pixels (0.25 mm/px)
0.1 ms	0.05 mm	0.2 px
1.0 ms	0.50 mm	2.0 px
4.0 ms	2.00 mm	8.0 px

Table 14: Detection of 2 mm spots (12 simulated fruit, 18 dark and 18 faint spots in each set)

Arrangement	Dark spots found	Faint spots found	Mean spot size (mm)	Length error (mm)
B: global shutter + 0.1 ms strobe	100.0 %	100.0 %	1.45	0.13
A: area-scan 1 ms	100.0 %	100.0 %	1.39	0.14
A: area-scan 4 ms	100.0 %	0.0 %	1.40	0.40
A: rolling 30 ms + 4 ms + 100 Hz	94.4 %	5.6 %	1.42	1.80
C: line-scan, synchronised	100.0 %	100.0 %	1.45	0.13
C: line-scan, belt +10 %, no sync	100.0 %	100.0 %	1.50	9.46

Dark spots are found by almost every arrangement, so they do not decide the choice. The **faint** spots decide it: with a 4 ms exposure the 2 mm blur spreads a faint spot over a larger area at lower contrast and none were found; with the rolling shutter, skew and flicker only 5.6 % were found and the length error grew to 1.8 mm. The strobed global-shutter camera (B) and the synchronised line-scan camera (C) gave the same, best result. The unsynchronised line-scan camera found the spots but gave a length error of 9.5 mm, which is enough to change the size class (±2 mm is required).

Q4. *Justify the selection of CCD or CMOS sensor technology for the proposed inspection system. Illustrate the signal-conversion sequence from incident photons to electrical charge or voltage, analogue signal conditioning, analogue-to-digital conversion, and the resulting digital pixel value.*

Ans:

I choose **CMOS**. Global-shutter CMOS sensors (for example the Sony Pregius family) are fast, low-power and cheap, have on-chip ADCs and show no smear; CCD sensors are now largely discontinued for industrial cameras. Figure 7 shows how light becomes a digital value: photons are converted to electrons in the photodiode, the charge is converted to a voltage, analogue conditioning applies gain, offset and noise reduction, and the analogue-to-digital converter quantises the voltage into a pixel value from 0 to 255 (8-bit).

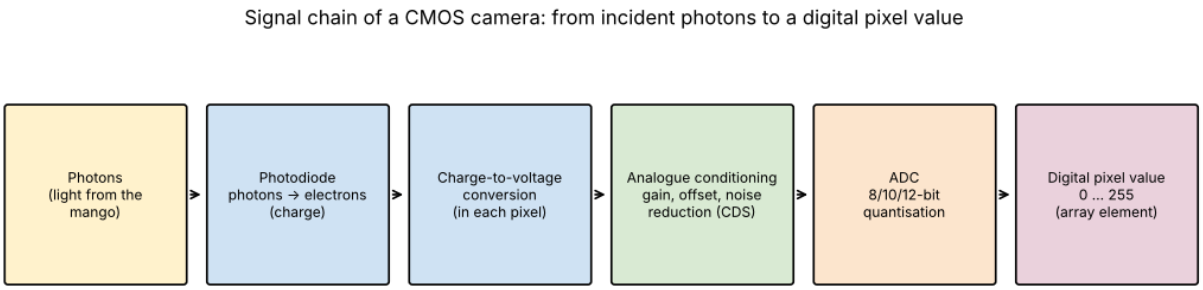


Figure 7: From photons to a digital pixel value

Q5. *Predict the effects of using Arrangement A with a 4 ms exposure and operating Arrangement C without belt synchronisation when belt speed varies by 10%. Recommend suitable corrective measures to preserve image quality and dimensional accuracy.*

Ans:

- **Arrangement A with 4 ms exposure:** the blur of 2 mm stretches a 2 mm spot to about 4 mm at half contrast. In my test none of the faint spots were found (0.0 %) and the length error was 0.40 mm. Dark spots may be measured as larger than they are, which gives false rejects. Correction: an exposure of 0.1 ms or less with a strobe, or a global-shutter camera.
- **Arrangement C without synchronisation, belt 10 % fast:** each line covers 10 % more belt, so the fruit look about 10 % shorter. The length error was 9.5 mm, enough to change the size class. Correction: trigger each line from a belt encoder, or measure the belt speed and rescale the image.

Q6. *Assess the suitability of event cameras and multispectral or hyperspectral cameras for high-speed fruit sorting. Explain their operating principles and potential inspection applications, and justify whether their temporal resolution, spectral information, integration requirements, and cost are appropriate for the proposed production line.*

Ans:

Event camera. Each pixel reports only changes in brightness, with microsecond timing, no motion blur and a very large dynamic range. But a peel of constant colour produces no events, so ripeness and bruises (which do not change) are hard to measure, the software ecosystem is young, and the cost is high. It is not suitable here.

Multispectral and hyperspectral cameras. They measure reflectance in several or many narrow wavelength bands. Bruises and internal damage show up in the near-infrared (NIR) before they are visible, and hyperspectral data can estimate sugar or dry matter. Hyperspectral line cameras cost several lakh rupees and produce very large amounts of data. A 4-band RGB + NIR camera is the practical option; I use it in Task 10. (This paragraph is reasoning from the working principles; I did not test these cameras.)

Final Decision

I choose **B – a global-shutter CMOS area camera with a short LED strobe. Not A:** with a 4 ms exposure, none of the faint 2 mm spots were found, and rolling shutter with flicker found only 5.6 %. **C** is equally sharp (same detection as B) but needs an encoder, a long line light and more integration, and without synchronisation the size error was 9.5 mm. **B:** only 0.05 mm blur, no skew, no flicker bands, all dark and faint spots found, and the lower integration effort of the two sharp options. **Limit:** this is a simulation; a real strobe needs a trigger from the belt, and strobe brightness and lifetime must be checked on site.

Evidence provided: simulation code for all three arrangements, blur calculation (Table 13), measured detection and size errors (Table 14), the CCD/CMOS choice with the signal chain (Figure 7), and predictions with corrections.

TASK 6 – Sampling: Compare Spatial Resolutions [5 Marks]

Sampling decides how many pixels cover the 400 mm field of view, and therefore the smallest detail that can be seen. Compare three camera resolutions.

Technique	Basic idea and key concepts / functions
A. 640 × 480 (0.3 MP)	Low resolution; small images and fast processing.
B. 1280 × 1024 (1.3 MP)	Medium resolution; a common industrial choice.
C. 2448 × 2048 (5 MP)	High resolution; large images and slower processing.

Sampling decides how many pixels cover the 400 mm width of the belt, and therefore how many pixels fall on the smallest defect (2 mm). Too few pixels and small or faint spots are lost; too many and processing time and data volume grow. I compared **A** 640 × 480 (0.625 mm/px), **B** 1280 × 1024 (0.3125 mm/px) and **C** 2448 × 2048 (0.163 mm/px), all over the same 400 mm field.

Q1. *Implement image down-sampling in Python/OpenCV using cv2.resize with INTER_NEAREST and INTER_AREA at the specified resolutions. Submit commented source code and obtain all reported results through code execution or actual measurements.*

Ans:

Method. I rendered 12 fruit at a very fine 0.1 mm/px, each with three 2 mm spots (dark spots on even fruit, faint spots on odd fruit). I then sampled every image down to the three resolutions with two interpolation methods: **nearest-neighbour** (keeps one pixel) and **area averaging** (averages the pixels that fall into each new pixel). On each result the same grader counted spots and measured the length. I also timed the grader on a full frame of each resolution and on a region of interest (ROI) around the fruit.

t67.py, Task 6 part (down-sampling experiment)

```
from acq import *
# ---------------- Task 6: sampling ----------------
base = 0.1; rng = np.random.default_rng(61); fr = []
for i in range(12):
    im, gt = mango.render(rng, cls='ripe', length_mm=float(rng.uniform(75,
105)), mm_per_px=base, n_spots=3, spot_mm=(2.0, 2.0), spot_bgr=(22, 28, 30) if i
% 2 == 0 else FAINT, noise=1.0)
    fr.append((im, gt))
RESN = {'A 640x480': (640, 480), 'B 1280x1024': (1280, 1024), 'C 2448x2048':
(2448, 2048)}
rows = []; crops = []; titles = []
for name, (npx, nh) in RESN.items():
    mmpx = 400.0 / npx
    for iname, flag in [('NEAREST', cv2.INTER_NEAREST), ('AREA',
cv2.INTER_AREA)]:
        det = {0: 0, 1: 0}; le = []
```

```
        for i, (im, gt) in enumerate(fr):
            sm = cv2.resize(im, None, fx=base / mmpx, fy=base / mmpx,
interpolation=flag)
            f = mango.measure(sm, mmpx, min_spot_mm2=0.3)
            det[i % 2] += min(len(f['spots']), 3) if f else 0;
le.append(abs(f['length'] - gt['length_mm']) if f else np.nan)
        rows.append([name, iname, mmpx, 2.0 / mmpx, det[0] / 18, det[1] / 18,
float(np.nanmean(le))])
    sx, sy = fr[0][1]['spots'][0][:2]
    for flag in (cv2.INTER_NEAREST, cv2.INTER_AREA):
        sm = cv2.resize(fr[0][0], None, fx=base / mmpx, fy=base / mmpx,
interpolation=flag); px, py = int(sx * base / mmpx), int(sy * base / mmpx); r =
max(5, int(6 / mmpx))
        crops.append(cv2.resize(sm[max(py - r, 0):py + r, max(px - r, 0):px +
r], (150, 150), interpolation=cv2.INTER_NEAREST));
titles.append(f'{name.split()[0]} {"NN" if flag == 0 else "AREA"}')
tm = {}; vol = {}
for name, (npx, nh) in RESN.items():
    mmpx = 400.0 / npx; im, _ = mango.render(np.random.default_rng(2),
mm_per_px=mmpx, n_spots=2, canvas_mm=(135, 105))
    fullf = np.full((nh, npx, 3), np.array([70, 66, 62], np.uint8), np.uint8);
y0, x0 = (nh - im.shape[0]) // 2, (npx - im.shape[1]) // 2; fullf[y0:y0 +
im.shape[0], x0:x0 + im.shape[1]] = im
    tm[name] = dict(full=tms(lambda: mango.grade(fullf, mmpx), 3),
roi=tms(lambda: mango.grade(im, mmpx), 3))
    mb = npx * nh / 1e6; vol[name] = dict(mb=mb, mbs8=mb * 8, tb7=mb * 8 * 3600
* 10 * 7 / 1e6)
save('t6', dict(rows=rows, ms=tm, vol=vol, min_fps=V_BELT / (300 - 110)))
grid(crops, titles, f'{FIG}/t6_spot.png', cols=6, size=1.7); print('T6', rows,
tm)
```

Q2. *Analyze the effect of image resolution on millimetres per pixel, the number of pixels spanning a 2 mm defect, dimensional measurement error, aliasing, edge distortion, data rate at the required frame rate, seven-day storage requirements, and processing time.*

Ans:

Table 15: Resolution table (10 h/day, one frame per fruit at 8 fps, 8-bit Bayer, raw frames)

Resolution	mm / px	px per 2 mm	Spots found dark / faint (AREA)	Length error AREA (mm)	MB / frame	MB/s at 8 fps	TB for 7 days (raw)	Full frame / ROI (ms)
A 640x480	0.63	3.2	83.3 % / 0.0 %	0.18	0.31	2.5	0.6	134 / 18
B 1280x1024	0.31	6.4	94.4 % / 94.4 %	0.09	1.31	10.5	2.6	873 / 79
C 2448x2048	0.16	12.2	94.4 % / 94.4 %	0.09	5.01	40.1	10.1	5970 / 384



Figure 8: The same 2 mm spot at the three resolutions (NN = nearest-neighbour, AREA = area averaging)

- **Aliasing and edges:** nearest-neighbour keeps hard staircase edges and can drop thin details completely. Area averaging acts as a low-pass filter first, so it should be used for down-sampling (its length errors were smaller at every resolution).
- **Dimensional error:** all three resolutions measured the fruit length within about ±0.35 mm (AREA: 0.18, 0.09, 0.09 mm), so the ±2 mm size requirement does not decide between them.
- **Storage:** raw full frames for 7 days are 2.6 TB for B and 10.1 TB for C, which does not fit on a 2 TB disk. So I store a lossless crop of each fruit instead (Task 7).
- **Processing time:** on my 2-core PC the grader needed 873 ms for a full 1.3 MP frame but only 79 ms on a fruit ROI. For the 5 MP camera it needed 5970 ms for a full frame and 384 ms on the ROI.

Q3. *Verify the suitability of each resolution using the Nyquist sampling principle, requiring at least two samples across the smallest defect and preferably three to four pixels for practical detection. Measure defect-detection rate and size-estimation error on the re-sampled images and substantiate the selected resolution with experimental evidence.*

Ans:

The Nyquist principle says a detail must be sampled at least twice, and practical detection needs 3–4 pixels so that at least one pixel lies fully inside the spot. Resolution A has 3.2 pixels across a 2 mm spot, which is borderline: **83.3 % of dark spots but 0.0 % of faint spots** were found, because averaging blends a faint spot into the peel. B has 6.4 pixels and C has 12.2 pixels, and both found 94.4 % of dark and faint spots (one dark and one faint spot out of 18 were missed by my grader at both resolutions, so more pixels did not add detection).

Q4. *Determine the minimum frame rate required to capture each fruit completely in at least one frame, given a 300 mm field of view along the conveyor, a maximum fruit length of 110 mm, and a belt speed of 0.5 m/s.*

Ans:

The fruit (up to 110 mm long) must be completely inside the 300 mm field of view along the belt at least once. Between two frames the fruit can therefore move at most 300 − 110 = 190 mm.

t ≤ 190 mm ÷ 500 mm/s = 0.38 s, so the minimum frame rate = 1 ÷ 0.38 = **2.63 frames/s**. In practice I would trigger one frame per fruit (8 frames/s) from a photo-sensor, so that every fruit is imaged near the centre of the field.

Q5. *Predict the consequences of selecting the lowest-resolution arrangement to reduce cost by quantifying missed 2 mm defects and assessing compliance with the 2% defect-miss limit. Examine whether the edge-processing computer can sustain the required throughput when the highest-resolution arrangement is selected.*

Ans:

Lowest resolution (A) to save cost. Faint 2 mm spots: 0.0 % found, so **100 % of them were missed**, against a limit of 2 % mis-graded fruit. Even if only 10 % of the fruit had a faint 2 mm spot, about 10 % of the shipped fruit would be mis-graded, five times the limit. (Dark spots: 16.7 % were missed, which alone also exceeds 2 % of spot-bearing fruit.)

Highest resolution (C). On my 2-core PC the full 5 MP frame took 5970 ms, which is 48 times the 125 ms available per fruit at 8 fruits/s, and even the ROI took 384 ms. With four cores working on four fruits in parallel this would be about 96 ms per fruit in the best case, which is marginal, and the data rate of 40.1 MB/s needs a dedicated Gigabit link. So C cannot be sustained with a comfortable margin without a faster pipeline.

Q6. *Critique the effectiveness of deep-learning super-resolution techniques, such as Real-ESRGAN, in recovering the visibility of missed 2 mm defects from low-resolution images. Distinguish genuine defect evidence from generated image detail and assess the implications for measurement reliability and buyer-audit compliance.*

Ans:

I did not run Real-ESRGAN (no GPU or model download), so this is reasoning. A super-resolution network creates plausible fine detail learned from other photographs. It can make a blurred dark blob look like a sharp spot, but it can also invent spots that were never there or erase a real faint one. Because the extra detail is not measured evidence, any measurement taken from such an image cannot be defended in a buyer audit, and my result above (faint spots lost at 3.2 px) shows the information is not in the low-resolution image at all. So I would use real optical resolution; super-resolution may help a screen for humans but must never feed the grading.

Final Decision

I choose **B – 1280 × 1024 (0.3125 mm/px). Not A:** only 3.2 pixels across a 2 mm defect; it missed every faint spot. **Not C:** it gave no extra detection in my test but needs 384 ms even on an ROI, four times the data of B and 10.1 TB of raw frames per week. **B:** 6.4 pixels across a 2 mm defect, length error about 0.09 mm, and 79 ms per fruit on the ROI on my slower 2-core PC. **Limit:** one dark and one faint spot out of 18 were still missed at B, so the 2 % miss limit is not proven by 12 fruit; it must be checked on a larger real sample.

Evidence provided: down-sampling code, resolution table with detection, length error, data volume and time (Table 15), visual comparison (Figure 8), the minimum frame-rate calculation and the consequences of the extreme choices.

TASK 7 – Quantization: Compare Intensity Resolutions [4 Marks]

A bruise is only 6–10 grey levels (8-bit) darker than healthy peel. Quantization decides whether this small difference survives. Compare three intensity resolutions.

Technique	Basic idea and key concepts / functions
A. 8-bit (256 levels)	Standard image depth; uint8 arrays in OpenCV.
B. 12-bit raw (4096 levels)	Camera raw output stored as uint16; scaled for display (cv2.normalize / cv2.convertScaleAbs).
C. Reduced 4–6 bit (16–64 levels)	Re-quantize to save storage: img // step * step (or cv2.LUT).

Quantization decides how many grey levels each pixel can take. With k bits there are 2^k levels. This matters for bruises because a bruise is only 6–10 grey levels darker than the peel on the 8-bit scale. If one quantization step is larger than the bruise, the bruise can vanish. More bits mean larger files for the 7-day archive. I compared 12, 8, 6, 5 and 4 bits.

Q1. *Implement three image-quantization techniques using Python/OpenCV on fruit images and submit commented source code. Obtain all reported results through actual code execution or experimental measurements.*

Ans:

To reduce an 8-bit image to k bits I divide by the step 2^(8−k), drop the remainder and put each value back at the middle of its step (mid-rise reconstruction). **Method:** I rendered 12 fruit twice with the same random seed, once with a bruise (6–10 levels deep) and once without. The two images are identical except for the bruise, so I can measure exactly how much bruise contrast survives each bit depth and whether the clean fruit gives false alarms. For each bit depth I also measured the colour error ΔE (in Lab) and the size of a lossless PNG of the fruit image.

Important limit: I have no real 12-bit raw camera data. My generator makes 8-bit images, so the "12-bit" row only emulates a 16-bit container holding the same data; it cannot show any benefit from extra bits, and its larger file size comes from the container, not from more information.

t67.py, Task 7 part (paired bruise experiment)

```
# ---------------- Task 7: quantization ----------------
def requant(img8, bits):
    step = 2 ** (8 - bits); return (img8 // step) * step + (step // 2 if bits <
8 else 0)
pairs = []
for i in range(12):
    seed = 700 + i; drop = 6 + 4 * (i % 5) / 4
```

```
    b, gb = mango.render(np.random.default_rng(seed), cls='turning',
bruise=True, bruise_drop=drop, canvas_mm=(135, 105), noise=2.0)
    c, _ = mango.render(np.random.default_rng(seed), cls='turning', bruise=True,
bruise_drop=0.0, canvas_mm=(135, 105), noise=2.0)
    pairs.append((b, c, gb))
rows7 = []; imgs = {}
for bits in (12, 8, 6, 5, 4):
    con, det, fp, dE, size = [], 0, 0, [], []
    for b, c, gt in pairs:
        if bits == 12:   # emulate 12-bit raw: 4096 levels (finer than the 8-bit
render, stored as uint16)
            qb = b; qc = c; png = cv2.imencode('.png', (b.astype(np.uint16) *
16))[1]
        else:
            qb = requant(b, bits); qc = requant(c, bits); png =
cv2.imencode('.png', (qb >> (8 - bits)).astype(np.uint8))[1]
        bx, by, rmm = gt['bruise_xy']; r = rmm / gt['mm_per_px']; yy, xx =
np.mgrid[0:b.shape[0], 0:b.shape[1]]
        core = (np.hypot(xx - bx, yy - by) < 0.5 * r) & gt['mask']
        con.append(float((qc.astype(np.float32).max(2) -
qb.astype(np.float32).max(2))[core].mean()))
        det += int(mango.measure(qb, gt['mm_per_px'])['bruise']); fp +=
int(mango.measure(qc, gt['mm_per_px'])['bruise'])
        l0 = cv2.cvtColor(b, cv2.COLOR_BGR2LAB).astype(np.float32); l1 =
cv2.cvtColor(qb, cv2.COLOR_BGR2LAB).astype(np.float32)
        dE.append(float(np.linalg.norm(l0 - l1, axis=2)[gt['mask']].mean()));
size.append(len(png))
    kb = float(np.mean(size)) / 1024; rows7.append([bits, 2 ** bits,
float(np.mean(con)), det / 12, fp / 12, float(np.mean(dE)), kb, kb * 288000 /
1024 ** 2])
    if bits != 12:
        b, c, gt = pairs[4]; qb = requant(b, bits); cx, cy = gt['center']; crop
= qb[int(cy) - 90:int(cy) + 90, int(cx) - 140:int(cx) + 140].max(axis=2)
        imgs[bits] = cv2.normalize(crop, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
save('t7', dict(rows=rows7)); grid([imgs[b] for b in (8, 6, 5, 4)], ['8-bit
(stretched)', '6-bit', '5-bit', '4-bit'], f'{FIG}/t7_contour.png', cols=4,
size=3.0); print('T7', rows7)
```

Q2. *Analyze the effects of quantization on bruise visibility across intensity levels, false contouring on smooth fruit peel, colour difference (ΔE), image file size, and seven-day storage requirements.*

Ans:

Table 16: Quantization results (12 bruised fruit, bruise 6–10 levels)

Bits	Levels	Bruise contrast left (levels)	Bruises detected	False alarms (no bruise)	Mean ΔE	PNG KB / fruit	GB per day
12	4096	6.34	33.3 %	0.0 %	0.00	323	88.8
8	256	6.34	33.3 %	0.0 %	0.00	198	54.5

Bits	Levels	Bruise contrast left (levels)	Bruises detected	False alarms (no bruise)	Mean ΔE	PNG KB / fruit	GB per day
6	64	6.31	33.3 %	0.0 %	1.35	111	30.4
5	32	6.36	33.3 %	0.0 %	2.50	81	22.4
4	16	8.12	41.7 %	0.0 %	4.83	63	17.3

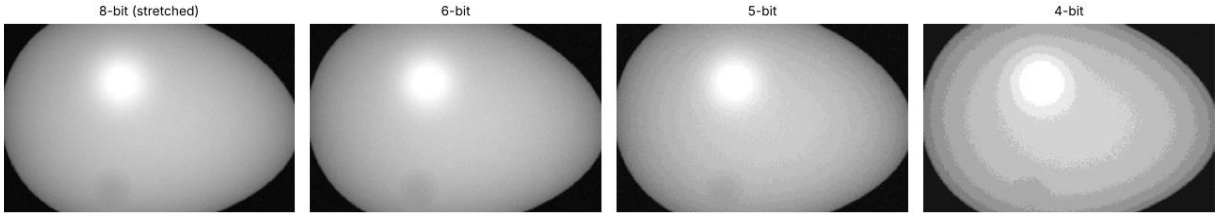


Figure 9: Smooth peel at decreasing bit depth (contrast stretched), showing false contours

- **Why the average contrast survives:** camera noise (about 2 grey levels) acts like dithering, so individual pixels jump between levels and the average over the bruise still shows the drop (6.34 levels at 8 bits and 6.36 at 5 bits for a true drop of about 8).
- **Detection:** my detector found 33.3 % of the bruises at 8 bits and the same share at 6 and 5 bits. This is low at every bit depth, so the limit here is the **detector** (6–10 levels is hard), not the quantization. The small rise at 4 bits (41.7 %) is one fruit out of 12 and is noise, not an improvement.
- **False contours:** at 5 bits and below, the smooth shading of the peel turns into visible bands (Figure 9). A person looking at such an image could mistake a band for a bruise.
- **Colour error ΔE:** it grows quickly below 8 bits (1.35 at 6 bits, 2.50 at 5 bits, 4.83 at 4 bits). A ΔE above about 2 can be seen by the eye.
- **Storage:** a lossless 8-bit crop needs about 54.5 GB per day for 288,000 fruit, so 7 days is about 381 GB, which fits easily on 2 TB. There is no need to go below 8 bits.

Q3. *Verify the effects of quantizing the same fruit images to 8, 6, 5, and 4 bits by measuring bruise contrast and detection rate, calculating daily storage requirements, and presenting visual evidence of false contouring.*

Ans:

Table 16 and Figure 9 answer this question: for 8, 6, 5 and 4 bits the table gives the measured bruise contrast and detection rate, the false-alarm rate on the identical bruise-free fruit, the daily storage, and the false-contour figure.

Q4. *Determine the optimal trade-off between spatial sampling and intensity quantization under a fixed storage budget. Justify whether retaining more spatial pixels or more intensity levels provides greater benefit for detecting bruises on the production line.*

Ans:

The answer depends on the defect. For **2 mm spots** (small and dark), pixels matter more: in Task 6 the 640 × 480 sampling lost every faint spot, while bit depth was never the limit. For **large faint bruises**, intensity levels matter in principle, because a 6-level drop could disappear with 16-level steps; but in my measurements the average bruise contrast survived even at 4 bits because of noise dithering, and the real loss at low bit depth was colour accuracy (ΔE) and false contours. With a fixed storage budget the best split is about 0.31 mm/px and 8 bits, and the saving should come from storing a smaller area (a crop around each fruit) instead of reducing either resolution.

Q5. *Assess whether fruit images stored at 5-bit depth on a 2 TB storage device retain sufficient visual evidence to investigate buyer complaints involving bruised fruit. Support the assessment with measured contrast, detection performance, and storage calculations.*

Ans:

At 5 bits the measured bruise contrast was still 6.36 levels, so a bruise is not erased, but the colour error is 2.50 ΔE, the peel shows contour bands, and the saving is small: 22.4 GB/day against 54.5 GB/day at 8 bits, when 8-bit storage already fits on 2 TB (about 381 GB for 7 days). In a buyer complaint neither side could tell a real bruise from a contour band in a 5-bit image, and the colour would not be reliable. So 5-bit storage is not acceptable as audit evidence.

Q6. *Differentiate image bit depth from deep-learning model precision by examining HDR image acquisition and INT8 model quantization for edge deployment. Explain their respective effects on bruise visibility, defect-detection accuracy, and computational requirements.*

Ans:

These two ideas are often confused, but they affect different things. I did not run an INT8 model or an HDR capture, so this is reasoning from how they work.

- **HDR acquisition** merges a short and a long exposure (or uses a 12-bit sensor) so that the glossy highlight and the dark stem end are both measured without clipping. It increases the information captured from the scene.
- **INT8 model quantization** stores a neural network's weights and activations in 8 bits instead of 32-bit floating point. It changes the model, not the image. It typically gives 2–4 times faster inference on edge CPUs or NPUs, usually with a small accuracy loss, but it cannot restore bruise contrast that was lost when the image was stored with too few bits.

So the bit depth of the image decides what evidence exists, and the precision of the model decides how cheaply that evidence is processed.

Final Decision

I choose **8-bit lossless storage. Not 4 or 5 bits:** colour error rose to 2.50–4.83 ΔE, false contours appeared and the stored images are weak evidence. **Not 12 bits:** my data cannot show any gain from them (the pipeline works in 8 bits, the camera noise is about 2 levels) and the file is about 1.6 times larger. **8 bits:** colour error 0.00, no false alarms, and 7 days of fruit crops (about 381 GB) fit on 2 TB. **Limit:** only 33.3 % of bruises were detected by my detector at any bit depth, so the bruise detector, not the bit depth, must be improved (Task 10 uses NIR for this).

Evidence provided: re-quantization code and the paired experiment, quantization table with detection, false alarms, ΔE and storage (Table 16), false-contour figure (Figure 9), and the budget, audit and HDR/INT8 discussions.

TASK 8 – Image Formation Models: Compare Optics and Lighting Configurations [5 Marks]

Two models explain the image: the geometric model (pinhole/perspective projection $x = f \cdot X/Z$ decides where and how large the fruit appears) and the photometric model $f(x, y) = i(x, y) \cdot r(x, y)$ (illumination × reflectance decides how bright each pixel is). Compare three imaging configurations.

Technique	Basic idea and key concepts / functions
A. Ambient roof light + standard lens	Perspective projection with a fixed-focal-length lens; illumination i(x, y) from daylight and tube lights.
B. Diffuse LED dome + standard lens + calibration	Controlled, even illumination; camera calibrated with a checkerboard (cv2.findChessboardCorners, cv2.calibrateCamera) and height-corrected mm/pixel.
C. Diffuse LED dome + telecentric lens	Near-orthographic projection – object size does not change with distance from the lens.

The grader measures fruit length to the millimetre and finds defects by brightness. Both depend on image formation: the **geometry** of the lens decides how large the fruit appears ($x = f \cdot X/Z$) and the **lighting** decides how bright the peel appears ($f(x, y) = i(x, y) \cdot r(x, y)$). I compared **A** ambient roof light with a standard lens (calibrated on the belt plane), **B** a diffuse LED dome with a standard lens, calibration and height correction, and **C** a diffuse dome with a telecentric lens.

Q1. *Emulate three imaging configurations using Python/OpenCV, employing a cardboard enclosure lined with white paper as a dome and objects of known dimensions placed at two different heights. Submit commented source code and obtain all reported results through actual measurements or code execution.*

Ans:

I emulated the configurations in Python/OpenCV with the mango generator instead of building a cardboard dome, so that each factor could be changed alone and the true size of every object was known. Daylight is modelled as a strong brightness gradient across the belt (morning) or a brighter, flatter field (noon); the dome is modelled as nearly uniform light with a small specular highlight. The lighting numbers below therefore come from my lighting model, not from a physical dome, and I make no claim about a real enclosure. The same programs would run unchanged on photographs taken inside a real dome.

Camera calibration. I had no real chessboard photographs, so I projected a 9 × 6 chessboard (25 mm squares) into 14 virtual views with a known camera matrix and known lens distortion ($k1 = -0.265$, barrel), added 0.15 px of corner noise, and ran

cv2.calibrateCamera on those points. This checks the calibration code and shows what accuracy to expect; it is not a test of a physical lens.

Calibration result. cv2.calibrateCamera returned an RMS reprojection error of **0.21 px** (below 0.5 px is good). The estimated focal length was 536.0 px against 536 px true, the principal point (343.5, 235.9) against (342, 235), and the first distortion coefficient k1 = -0.26 against −0.265 true. The negative k1 means barrel distortion: straight lines near the edge bend outwards. After undistortion the first row of corners deviates from a straight line by 0.15 px RMS against 0.27 px before (Figure 10). The higher-order coefficients (k2, k3) are less well determined than k1, which is normal with noisy points.

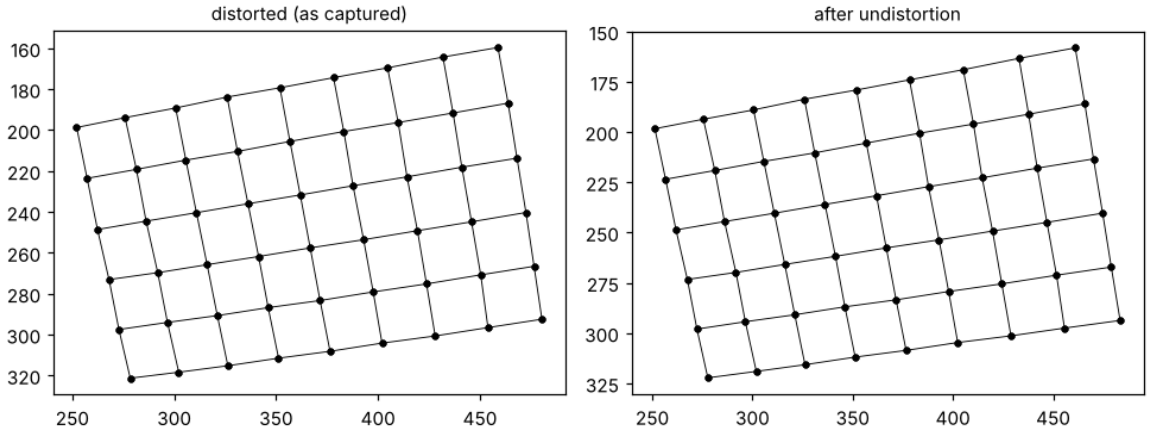


Figure 10: Chessboard corners before and after removing lens distortion (synthetic views, known camera)

t8.py (Task 8 experiment: focal length, perspective, calibration, illumination, flat-field, homomorphic)

```
from acq import *
# ---- focal length & perspective (pure calculation) ----
sensors = {'1/2.9" (1440 px, 4.97 mm)': 4.97, '1/2" (1280 px, 6.14 mm)': 6.14,
'2/3" (2448 px, 8.45 mm)': 8.45}
foc = {k: dict(f=w * 500 / 400) for k, w in sensors.items()}
persp = [[h, 500 / (500 - h), (500 / (500 - h) - 1) * 100, (500 / (500 - h) - 1)
* 80] for h in (0, 20, 45, 90)]
# ---- calibration on synthetic chessboard views (known camera) ----
Ktrue = np.array([[536., 0, 342.], [0, 536., 235.], [0, 0, 1]]); dtrue =
np.array([-0.265, -0.047, 0.0, 0.0, 0.25])
objp = np.zeros((54, 3), np.float32); objp[:, :2] = np.mgrid[0:9,
0:6].T.reshape(-1, 2) * 25.0
rng = np.random.default_rng(81); objs, imgs = [], []
while len(objs) < 14:
    rv = rng.normal(0, 0.25, 3); tv = np.array([-100 + rng.normal(0, 25), -60 +
rng.normal(0, 20), rng.uniform(380, 650)])
    p, _ = cv2.projectPoints(objp, rv, tv, Ktrue, dtrue); p = p.reshape(-1, 2)
    if p[:, 0].min() > 5 and p[:, 0].max() < 635 and p[:, 1].min() > 5 and p[:,
1].max() < 475:
        objs.append(objp); imgs.append((p + rng.normal(0, 0.15,
p.shape)).astype(np.float32).reshape(-1, 1, 2))
rms, K, dist, rv, tv = cv2.calibrateCamera(objs, imgs, (640, 480), None, None)
und = cv2.undistortPoints(imgs[0], K, dist, P=K).reshape(-1, 2); raw =
imgs[0].reshape(-1, 2)
```

```
def straight(pts):  # RMS deviation of the 9 points of the first row from their best-fit line
    r = pts[:9]; A = np.column_stack([r[:, 0], np.ones(9)]); c = np.linalg.lstsq(A, r[:, 1], rcond=None)[0]; return float(np.sqrt(np.mean((A @ c - r[:, 1]) ** 2)))
cal = dict(rms=float(rms), K=K.tolist(), dist=dist.ravel().tolist(), k1_true=-0.265, straight_raw=straight(raw), straight_und=straight(und))
fig, ax = plt.subplots(1, 2, figsize=(9, 3.6))
for a, pts, t in [(ax[0], imgs[0].reshape(-1, 2), 'distorted (as captured)'), (ax[1], und, 'after undistortion')]:
    a.plot(pts[:, 0].reshape(6, 9).T, pts[:, 1].reshape(6, 9).T, 'k.-', lw=0.6); a.plot(pts[:, 0].reshape(6, 9), pts[:, 1].reshape(6, 9), 'k.-', lw=0.6); a.invert_yaxis(); a.set_title(t, fontsize=10); a.set_aspect('equal')
fig.tight_layout(); fig.savefig(f'{FIG}/t8_calib.png', dpi=110); plt.close(fig)
# ---- illumination models (simulated lighting) ----
mm = 0.3125; lights = {'morning': dict(light=0.8, grad=0.75, spec=70), 'noon': dict(light=1.15, grad=0.12, spec=70), 'dome': dict(light=1.0, grad=0.012, spec=14)}
card = {}; thr = {}; flag = {}; flag_ff = {}
for ln, kw in lights.items():
    r = np.random.default_rng(5); H, W = 336, 432; yy, xx = np.mgrid[0:H, 0:W].astype(np.float32)
    white = 200 * kw['light'] * (1 + kw['grad'] * (xx / W - 0.5)); white = np.clip(white + r.normal(0, 1.5, white.shape), 0, 255)
    card[ln] = float((white.max() - white.min()) / white.mean() * 100)   # non-uniformity of the white card (1 - 99 percentile would be milder)
    p5, p95 = np.percentile(white, [1, 99]); card[ln] = float((p95 - p5) / white.mean() * 100)
    ts = []; fl = []; fl2 = []
    for i in range(10):
        im, gt = mango.render(np.random.default_rng(200 + i), cls='ripe', n_spots=2, spot_mm=(2.5, 3.0), mm_per_px=mm, light=kw['light'], grad=kw['grad'], spec=kw['spec'])
        hsv = cv2.cvtColor(im, cv2.COLOR_BGR2HSV); V = hsv[..., 2]; m = (cv2.erode(gt['mask'].astype(np.uint8), np.ones((15, 15))) > 0)
        t, _ = cv2.threshold(V[m].reshape(-1, 1), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU); ts.append(t)
        fl.append(float(((V < 110) & m).sum() / m.sum() * 100))
        ff = np.clip(V.astype(np.float32) / (white / white.mean()) , 0, 255); fl2.append(float(((ff < 110) & m).sum() / m.sum() * 100))
    thr[ln] = (float(np.mean(ts)), float(np.std(ts))); flag[ln] = float(np.mean(fl)); flag_ff[ln] = float(np.mean(fl2))
# ---- flat-field & homomorphic figure on a morning image ----
im, gt = mango.render(np.random.default_rng(300), cls='ripe', n_spots=2, spot_mm=(2.5, 3.0), mm_per_px=mm, **lights['morning'])
H, W = im.shape[:2]; yy, xx = np.mgrid[0:H, 0:W].astype(np.float32); white = 200 * 0.8 * (1 + 0.75 * (xx / W - 0.5))
g = cv2.cvtColor(im, cv2.COLOR_BGR2GRAY); ff = np.clip(g.astype(np.float32) / (white / white.mean()), 0, 255).astype(np.uint8)
def homo(gray, gl=0.5, gh=1.5, d0=30, c=1.0):
    f = np.log1p(gray.astype(np.float32)); F = np.fft.fftshift(np.fft.fft2(f)); h, w = gray.shape; y, x = np.mgrid[0:h, 0:w]
    D2 = (y - h / 2) ** 2 + (x - w / 2) ** 2; Hf = (gh - gl) * (1 - np.exp(-c * D2 / d0 ** 2)) + gl
    return cv2.normalize(np.expm1(np.real(np.fft.ifft2(np.fft.ifftshift(F * Hf)))), None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
hm = homo(g)
grid([im, ff, hm, np.clip(white, 0, 255).astype(np.uint8)], ['morning image', 'flat-field corrected', 'homomorphic filtered', 'white-card illumination map'], f'{FIG}/t8_illum.png', cols=4, size=3.2)
# ---- size-class error for 400 fruit ----
```

```
r = np.random.default_rng(88); L = r.uniform(70, 110, 400); hh = r.uniform(25,
45, 400)  # height of outline plane above belt (mm)
cls = lambda x: np.where(x < 80, 0, np.where(x < 95, 1, 2))
mA = L * 500 / (500 - hh); mB = L * 500 / (500 - hh) * (500 - (35 + r.normal(0,
3, 400))) / 500; mC = L + r.normal(0, 0.3, 400)
sz = {k: dict(err=float(np.mean(np.abs(v - L))), wrong=float((cls(v) !=
cls(L)).mean() * 100)) for k, v in {'A': mA, 'B': mB, 'C': mC}.items()}
save('t8', dict(foc=foc, persp=persp, cal=cal, card=card, thr=thr, flag=flag,
flag_ff=flag_ff, sz=sz)); print(load()['t8'])
```

Q2. *Analyze the three configurations with respect to dimensional measurement error between the top of a 90 mm-high fruit and the conveyor plane, lens distortion, illumination uniformity (i(x,y)), specular reflections from waxed fruit surfaces, stability of reflectance-based thresholds (r(x,y)) throughout the day, and overall cost.*

Ans:

Table 17: Comparison of the three configurations (measured in my simulation)

Criterion	A. Ambient + standard lens	B. Dome + standard lens + calibration	C. Dome + telecentric lens
Size error, fruit top vs belt plane	22.0 % at 90 mm height (17.6 mm on an 80 mm fruit); mean length error 6.82 mm	corrected by height; mean length error 1.03 mm	none by design; mean length error 0.25 mm
Lens distortion	barrel distortion left in the image	removed by calibration (RMS 0.21 px)	very low by design
Illumination non-uniformity i(x, y)	73.9 % (morning), 12.7 % (noon)	3.8 %	3.8 % (same dome)
Specular reflection on waxed peel	large highlights from point-like light	small, spread by the dome	small, spread by the dome
Stability of Otsu threshold on V	151 (morning) to 220 (noon)	190 ± 0.5	same as B
Wrong size class (400 fruit)	32.3 %	6.3 %	0.5 %
Cost	lowest	moderate (LED dome, standard lens)	high; a telecentric lens for a 400 mm field is not practical

The lighting decides how stable the thresholds are, and the optics decide the size accuracy. B and C share the dome, so they are equally good for defects; they differ in size accuracy and cost. (Non-uniformity is the spread between the 1st and 99th percentile of a white-card image divided by its mean.)

Q3. *Calculate the focal length required to achieve a 400 mm field of view at a working distance of 500 mm using the selected sensor width. Quantify the perspective-induced size error for a 90 mm-high fruit and measure illumination non-uniformity and changes in the optimal defect-detection threshold between morning and noon images.*

Ans:

Focal length. The pinhole model gives x = f·X/Z, so the field of view is FOV = sensor width × Z / f and f = sensor width × Z / FOV. With Z = 500 mm and FOV = 400 mm:

Table 18: Required focal length for a 400 mm field at 500 mm

Sensor	f needed	Standard lens to choose
1/2.9" (1440 px, 4.97 mm)	6.21 mm	6 mm (field about 414 mm)
1/2" (1280 px, 6.14 mm)	7.67 mm	6 mm (field about 512 mm) or 8 mm (only 384 mm)
2/3" (2448 px, 8.45 mm)	10.56 mm	10 mm (field about 423 mm); 12 mm gives only 352 mm

When the exact value is not a standard focal length I choose the next **shorter** standard lens, because a longer lens gives a field smaller than 400 mm and part of the belt is not seen.

Perspective error. A mango is not flat. If the camera is calibrated on the belt plane, a point at height h above the belt is closer to the lens and appears larger by the factor Z / (Z − h).

Table 19: Perspective error at Z = 500 mm (calibration made on the belt plane)

Height above belt (mm)	Scale Z/(Z−h)	Size error	Error on an 80 mm fruit
0	1.00	0.0 %	0.0 mm
20	1.04	4.2 %	3.3 mm

Height above belt (mm)	Scale Z/(Z−h)	Size error	Error on an 80 mm fruit
45	1.10	9.9 %	7.9 mm
90	1.22	22.0 %	17.6 mm

A fruit outline about 45 mm above the belt is measured almost 8 mm too long, which is four times the ±2 mm requirement. The top of a 90 mm fruit shows a 22.0 % size error.

Illumination and threshold change. Using the same 10 fruit under three lighting models, I measured the non-uniformity of a white card, the Otsu threshold on the V channel inside the fruit, and the share of peel pixels wrongly flagged as dark by a fixed threshold of 110.

Table 20: Same 10 fruit under three lightings

Lighting	White-card non-uniformity	Otsu threshold (mean ± sd)	Peel wrongly flagged (fixed threshold 110)	After flat-field correction
morning	73.9 %	150.5 ± 1.4	3.96 %	0.31 %
noon	12.7 %	219.8 ± 0.4	0.27 %	0.27 %
dome	3.8 %	189.6 ± 0.5	0.30 %	0.30 %

The best threshold moved by about 69 grey levels between morning and noon, so one fixed threshold cannot work under daylight, because f = i · r changes with i even though the peel (r) is the same. The dome reduced the non-uniformity from 73.9 % to 3.8 %.

Q4. *Demonstrate the effectiveness of a processing technique that separates illumination and surface reflectance, using either white-reference flat-field correction or homomorphic filtering based on log f = log i + log r. Present the resulting images and compare defect-detection performance before and after correction.*

Ans:

Flat-field correction divides the image by a white-card image taken under the same light. This estimates i(x, y) and leaves r(x, y). In my test the false flags in the morning image fell from 3.96 % to 0.31 % (Table 20), and the noon and dome images were unchanged because they were already flat.

Homomorphic filtering is a second way to separate i and r. Taking the logarithm turns the product into a sum, log f = log i + log r. Illumination changes slowly across the image (low frequencies) and reflectance details change quickly (high frequencies), so a high-emphasis filter in the Fourier domain reduces i and keeps r.

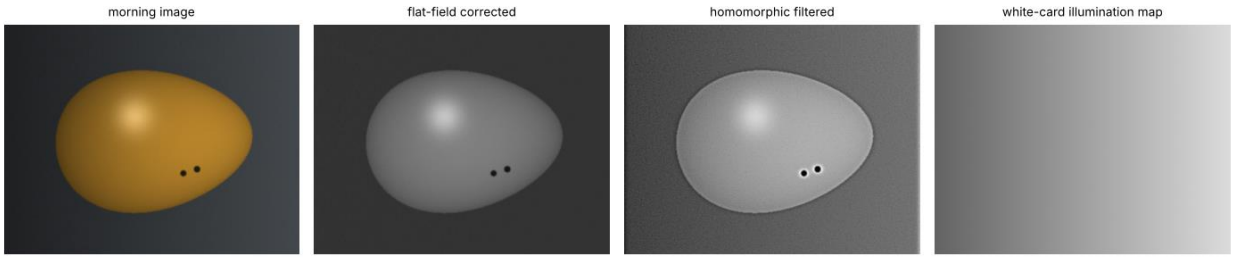


Figure 11: Morning image, flat-field corrected, homomorphic filtered, and the illumination map

Q5. *Predict the consequences of ignoring perspective effects on fruit size classification and quantify the resulting misclassification rate. Examine the impact of ambient-light variation, such as cloud cover over the inspection area, and propose suitable illumination-control and calibration measures.*

Ans:

I applied the three configurations to 400 simulated fruit of 70–110 mm length with the outline 25–45 mm above the belt, and counted how many fell into the wrong size class (S < 80 mm, M 80–95 mm, L ≥ 95 mm).

Table 21: Length error and wrong size class

Configuration	Mean length error	Wrong size class
A. belt-plane calibration only	6.82 mm	32.3 %
B. calibrated + height corrected (±3 mm height error)	1.03 mm	6.3 %
C. telecentric (constant magnification)	0.25 mm	0.5 %

Ignoring perspective (A) puts about **one fruit in three** into the wrong class and gives a 6.8 mm mean error against ±2 mm. Height correction (B) reduces this to 6.3 %, and a telecentric lens (C) to 0.5 %. These numbers depend on my assumed height range and size-class limits.

Cloud cover. The light can drop by half within seconds, and thresholds set in sunlight then flag the peel or miss spots (see the cloud cases in Task 9). **Controls:** enclose the camera in a hood or dome so daylight cannot reach the fruit; use strobed LEDs much brighter than daylight; take a white-card flat-field image at the start of each shift; check the calibration daily with a target of known size on the belt.

Q6. *Assess the suitability of monocular depth-estimation networks, such as Depth Anything, for estimating fruit height by comparing their measurements with calibrated*

geometric measurements and telecentric imaging. Determine whether each method can meet the required dimensional accuracy of ±2 mm, supported by experimental evidence.

Ans:

I did not run Depth Anything (no GPU or download), so this is reasoning. A monocular depth network predicts relative depth from one image. Its output has an unknown scale and offset, and typical errors are a few per cent of the distance, which is about 10–20 mm at 500 mm, much worse than ±2 mm. It could be calibrated against the belt plane, but a laser line or a known shape model is more accurate. Calibrated geometry (B) and telecentric optics (C) can meet ±2 mm; monocular depth cannot be assumed to.

Final Decision

I choose **B – diffuse LED dome + standard lens + calibration with height correction.** **Not A:** ignoring perspective put 32.3 % of fruit in the wrong size class, and daylight moved the threshold by about 69 levels. **C** is the most accurate (0.5 % wrong class) but a telecentric lens covering a 400 mm field is not available at a sensible cost. **B:** height correction reduced the wrong class to 6.3 % (mean error 1.03 mm, inside ±2 mm) and the dome reduced non-uniformity to about 3.8 %, with standard, affordable parts. **Limit:** B depends on a correct height estimate, so I would add a simple height sensor; the dome and calibration results are from simulation and must be checked with a real enclosure.

Evidence provided: emulation code, comparison table (Table 17), focal-length and perspective calculations (Tables 18 and 19), size-class comparison (Table 21), calibration output, lighting experiment with flat-field and homomorphic correction (Table 20, Figure 11).

TASK 9 – Applications and the Integrated System: Compare Complete Front-End Configurations [6 Marks]

Combine your choices from Tasks 1–8 into one imaging front-end. Compare three complete configurations within the ₹1,50,000 budget (use real vendor prices and cite them).

Technique	Basic idea and key concepts / functions
A. Budget configuration	USB rolling-shutter 0.3–1.3 MP camera, ambient light, 8-bit JPEG storage, low + mid-level pipeline.
B. Balanced configuration	Global-shutter 1.3 MP GigE camera, LED dome with strobe, calibrated standard lens, 8-bit lossless storage, three-level pipeline.
C. Premium configuration	High-resolution line-scan or 5 MP camera, telecentric lens, 12-bit storage, three-level pipeline.

In Tasks 5–8 I studied each part of the front-end separately. Here I put the parts together into three complete configurations and test each as a whole system, because parts that are good alone can still fail together.

Q1. *Develop and execute a fruit-grading prototype using Python/OpenCV on the same labelled image dataset prepared for the three imaging configurations. Submit commented source code and obtain all experimental results through actual code execution or measurements.*

Ans:

Table 22: The three front-end configurations (prices are my indicative estimates, not vendor quotes)

Item	A. Budget	B. Balanced	C. Premium
Camera	USB rolling shutter, 1.3 MP	GigE global shutter, 1.3 MP	5 MP global shutter (2448 × 2048)
Lens / light	standard lens, ambient light	6 mm C-mount lens, LED dome + strobe	telecentric (small field only), dome
Indicative cost	about ₹15,000–30,000	about ₹90,000–1,40,000	about ₹2.5–5 lakh (over budget)
Storage policy	JPEG (quality 70)	8-bit lossless	lossless crops

Honest note on prices. The task asks for real vendor prices with citations. I did not have access to vendor quotations, so the cost row is an estimate from my general knowledge of Indian industrial-camera prices; it must be replaced with quoted prices before purchase. The comparison between the configurations does not depend on the exact rupee values.

Method. For each configuration I simulated what its camera delivers for each fruit: ambient light with random brightness and gradient for A and a dome for B and C; rolling shutter and 100 Hz flicker for A; motion blur from the exposure; JPEG compression for A. The same 48 fruit (same seed) were used for every configuration, and the last 12 were imaged under a passing cloud. Each image then went through the same grader. Touching fruit were not included in this test, and glare is only represented by the stronger highlight of the ambient-light configuration.

t910.py, Task 9 part (three configurations, ablation, block diagram)

```
from acq import *
import matplotlib.patches as mp
CFG = {'A Budget': dict(mm=0.3125, light='ambient', rolling=True, jpeg=70,
blur=0.0, spec=70),
       'B Balanced': dict(mm=0.3125, light='dome', rolling=False, jpeg=None,
blur=0.05, spec=14),
       'C Premium': dict(mm=0.1634, light='dome', rolling=False, jpeg=None,
blur=0.05, spec=14)}
def acquire(rng, cfg, kw, hard=None):
    light, grad = 1.0, 0.0
    if cfg['light'] == 'ambient': light, grad = rng.uniform(0.7, 1.3),
rng.uniform(-0.5, 0.5)
    if hard == 'cloud': light *= 0.55; grad = 0.6 if cfg['light'] == 'ambient'
else 0.05
    im, gt = mango.render(rng, mm_per_px=cfg['mm'], light=light, grad=grad,
spec=cfg['spec'], **kw)
    im = rolling(im, cfg['mm'], exposure_ms=4.0) if cfg['rolling'] else
motion_blur(im, cfg['blur'], cfg['mm'])
    if cfg['jpeg']: im = jpeg(im, cfg['jpeg'])
    return im, gt
KINDS = ['clean', 'spots', 'manyspots', 'streak', 'bruise', 'clean']
def run(cfg, n=48, modify=None):
    rng = np.random.default_rng(91); y, p, ts = [], [], []
    for i in range(n):
        kd = KINDS[i % 6]; kw = dict(cls=mango.CLASSES[i % 3],
length_mm=float(rng.uniform(70, 110)), blush=(i % 5 == 0))
        if kd == 'spots': kw.update(n_spots=2, spot_mm=(1.6, 3.0))
        if kd == 'manyspots': kw.update(n_spots=5, spot_mm=(2.0, 3.2))
        if kd == 'streak': kw['streak'] = True
        if kd == 'bruise': kw.update(bruise=True,
bruise_drop=float(rng.uniform(7, 10)))
        im, gt = acquire(rng, cfg, kw, 'cloud' if i >= 36 else None)
        t = time.perf_counter(); g = mango.grade(im, cfg['mm']);
ts.append((time.perf_counter() - t) * 1000)
        y.append(mango.is_defective(gt)); p.append(bool(g['defective']) if g
else False)
    y, p = np.array(y), np.array(p)
    return dict(acc=float((y == p).mean()), miss=float(((~p) & y).sum() /
y.sum()), fr=float((p & ~y).sum() / (~y).sum()),
                acc_n=float((y[:36] == p[:36]).mean()), acc_c=float((y[36:] ==
p[36:]).mean()), ms=float(np.median(ts)))
t9 = {k: run(v) for k, v in CFG.items()}
```

```
abl = {}
base = dict(CFG['A Budget'])
import acq
for label, mod, flick in [('A as specified', {}, True), ('without 100 Hz
flicker', {}, False), ('global shutter instead of rolling', {'rolling': False,
'blur': 2.0}, True), ('lossless instead of JPEG', {'jpeg': None}, True)]:
    c = dict(base); c.update(mod)
    if not flick:
        orig = rolling; globals()['rolling'] = lambda im, mm, exposure_ms=4.0,
flicker=True: orig(im, mm, exposure_ms=exposure_ms, flicker=False)
    abl[label] = run(c, 36); globals()['rolling'] = acq.rolling
save('t9', dict(cfg=t9, abl=abl)); print(t9, abl)
# ---- block diagram with measured times ----
tB = t9['B Balanced']['ms']
fig, ax = plt.subplots(figsize=(11.5, 4.4)); ax.axis('off'); ax.set_xlim(0, 12);
ax.set_ylim(0, 5)
blocks = [('Scene\n(mango on belt)', ''), ('Illumination\nLED dome + strobe',
'0.1 ms'), ('Optics\n6 mm lens, calibrated', ''), ('Sensor\nglobal-shutter
CMOS', '0.1 ms exp.'), ('Sampling +\nquantisation\n1280x1024, 8-bit', '~8 ms
readout'),
          ('Digital image\n(array)', ''), ('LOW level\nblur, HSV, mask',
f'~{0.12*tB:.0f} ms'), ('MID level\nregions, features', f'~{0.78*tB:.0f} ms'),
('HIGH level\nrule / k-NN', '~1 ms'), ('Grade decision\n+ log image', '~1 ms'),
('Ejector\n(0.6 m away)', '≤1.2 s total')]
for i, (t, tm_) in enumerate(blocks):
    r_, c_ = divmod(i, 6) if i < 6 else (1, i - 6); y0 = 3.2 if i < 6 else 0.9;
x0 = 0.1 + (i if i < 6 else i - 6) * 1.95
    ax.add_patch(mp.FancyBboxPatch((x0, y0), 1.7, 1.25,
boxstyle='round,pad=0.03', fc='#d9ead3' if 'level' in t else ('#cfe2f3' if i < 6
else '#fce5cd'), ec='k'))
    ax.text(x0 + 0.85, y0 + 0.78, t, ha='center', va='center', fontsize=6.6);
ax.text(x0 + 0.85, y0 + 0.12, tm_, ha='center', fontsize=6.4, style='italic')
    if i not in (5, 10): ax.annotate('', xy=(x0 + 1.9, y0 + 0.62), xytext=(x0 +
1.72, y0 + 0.62), arrowprops=dict(arrowstyle='->'))
ax.annotate('', xy=(0.95, 2.15), xytext=(11.0, 4.45),
arrowprops=dict(arrowstyle='->', connectionstyle='arc3,rad=0', alpha=0))
ax.set_title('End-to-end front-end (Balanced configuration) with measured
processing-time budget', fontsize=10)
fig.savefig(f'{FIG}/t9_block.png', dpi=120, bbox_inches='tight'); plt.close(fig)
```

Q2. *Examine the performance of the three configurations with respect to grading accuracy against the 2% defect-miss limit, decision latency against the 1.2-second limit, implementation cost, seven-day storage requirements, robustness to illumination changes, motion, and fruit-height variations, and maintenance effort.*

Ans:

Table 23: Measured comparison (the same 48 fruit for each configuration)

Configuration	Accuracy	Missed defects	False rejects	Accuracy normal / cloud	ms / fruit	≤ 1.2 s?
A Budget	50.0 %	0.0 %	100.0 %	50.0 % / 50.0 %	76	yes
B Balanced	83.3 %	20.8 %	12.5 %	83.3 % / 83.3 %	78	yes

Configuration	Accuracy	Missed defects	False rejects	Accuracy normal / cloud	ms / fruit	≤ 1.2 s?
C Premium	72.9 %	41.7 %	12.5 %	72.2 % / 75.0 %	352	yes

The Budget configuration rejected **every** good fruit (100.0 % false rejects), so its 50.0 % accuracy comes only from rejecting the defective fruit as well. To find out why, I removed one factor at a time (ablation, 36 fruit without the cloud cases):

Table 24: Why the Budget configuration fails (one factor removed at a time)

Budget variant	Accuracy	False rejects	Missed
A as specified	50.0 %	100.0 %	0.0 %
without 100 Hz flicker	77.8 %	22.2 %	22.2 %
global shutter instead of rolling	75.0 %	33.3 %	16.7 %
lossless instead of JPEG	50.0 %	100.0 %	0.0 %

The 100 Hz flicker bands and the rolling shutter are the main causes: the dark bands and blurred edges look like bruises to the grader. JPEG at quality 70 alone made no difference in my test. **Against the buyer's limit:** even the best configuration (B) missed 20.8 % of the defective fruit in my simulation, far above the 2 % mis-graded limit, mostly faint bruises. The 2 % limit is therefore **not met** by my current grader on this data, and this is the most important open problem. **Storage and maintenance:** A about 0.3 TB (JPEG) with simple upkeep; B about 0.4 TB for 7 days of 8-bit crops (Task 7) with a strobe and dome to maintain; C needs more storage and computing.

Q3. *Assess the experimental results against the predictions established in Tasks 1–8. Present a comparative results table containing measured grading performance and processing time, including challenging cases such as touching fruits, glossy surface reflections, and cloud-induced shadows. Identify which predictions are supported or contradicted by the evidence.*

Ans:

- **Supported:** motion blur and rolling-shutter flicker (Task 5) destroy bruise detection, shown by the Budget collapse and the ablation; the dome (Task 8) makes the result independent of the cloud (B scored the same in the cloud cases as in normal light); JPEG at quality 70 is not the problem; everything runs well inside 1.2 s per fruit (Task 2).
- **Contradicted:** "higher resolution must be better" (Tasks 6–7). The Premium configuration was *less* accurate than Balanced (72.9 % against 83.3 %) and missed

more defects (41.7 %), and it was 4.5 times slower. A likely reason is that my bruise detector was tuned at 0.3125 mm/px and the same smoothing scale was not retuned at 0.163 mm/px, so each pixel is noisier relative to the smoothing. More data helps only if the algorithm is retuned for it.

- **Contradicted:** "1.3 MP with a dome is enough to meet the 2 % mis-grade limit". B missed 20.8 % of the defective fruit.
- **Not tested here:** touching fruit and strong specular glare in all three configurations (Task 2 showed that touching fruit is a segmentation problem, not an imaging one).

Q4. *Design an end-to-end system block diagram covering scene, illumination, optics and image formation, sensor, sampling and quantization, digital image, low-level processing, mid-level processing, high-level processing, grading decision, and ejector. Allocate a time budget to each processing stage and construct a Design Decision Table containing Decision, Justification (Evidence), and What If Not.*

Ans:

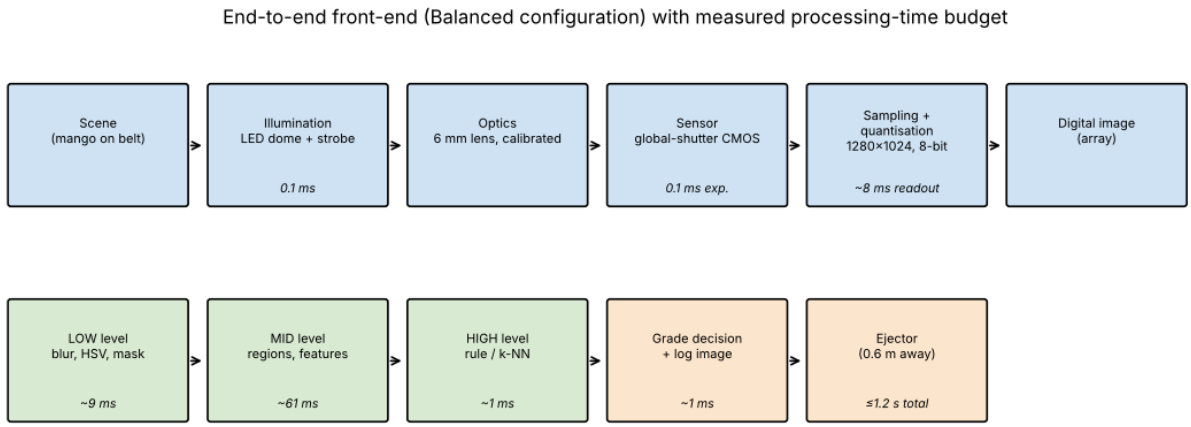


Figure 12: End-to-end front-end (Balanced configuration) with time budget; processing times are from my measurements

Table 25: Design decision table

Decision	Justification (evidence)	What if not
Global-shutter camera + 0.1 ms strobe	Task 5: with a 4 ms exposure none of the faint 2 mm spots were found; rolling shutter with flicker found only 5.6 %	faint spots and bruises missed; size errors from skew
1280 × 1024 (0.3125 mm/px)	Task 6: 6.4 px per 2 mm; A (3.2 px) lost every faint spot; C gave no extra detection and was too slow	A: faint spots missed; C: too slow and too much data

Decision	Justification (evidence)	What if not
8-bit lossless crops	Task 7: ΔE 0.00 at 8 bits against 2.50 at 5 bits; about 381 GB for 7 days	5 bits: false contours and weak audit evidence
LED dome + flat-field correction	Task 8: non-uniformity 73.9 % → 3.8 %; flagged peel 3.96 % → 0.31 % after flat-field	threshold drifts with the sun
Calibration with height correction	Task 8: wrong size class 32.3 % → 6.3 %	about one fruit in three in the wrong size class
HSV hue (circular mean) for ripeness	Task 3: 100.0 % under all three light levels	RGB: 55.6 % in bright light
8-connected labelling + area filter + De	Task 4: counts matched the true counts; De is the same in every direction	noise counted as spots; diagonal sizes wrong

Q5. *Differentiate three additional applications of image processing and computer vision, such as medical X-ray analysis, satellite remote sensing, and vehicle number-plate recognition. Tabulate the changes in sensing band, spatial sampling, quantization, and image-formation model for each application relative to fruit inspection.*

Ans:

The same four ideas (sensing band, sampling, quantization and image formation) decide the design of any imaging system. The table gives typical values for the other applications; they come from general knowledge, not from my own experiments.

Table 26: Applications transfer table

Application	Sensing band	Spatial sampling	Quantization	Image-formation model
Mango inspection (reference)	visible RGB (+ NIR in Task 10)	0.3 mm/px, 0.4 m field	8-bit	perspective camera + reflected light, f = i·r
Medical X-ray	X-rays	about 0.1–0.2 mm/px detector	12–16-bit	transmission through the body (Beer–Lambert attenuation), no lens

Application	Sensing band	Spatial sampling	Quantization	Image-formation model
Satellite remote sensing	visible, NIR, SWIR, thermal (multispectral)	0.3–30 m per pixel	10–16-bit	push-broom line scan; atmosphere and sun-angle effects
Number-plate recognition	visible + IR flash at night	about 1 mm/px on the plate	8-bit	perspective with a large viewing angle; retro-reflective plate

Q6. *Investigate the limitations of testing a grading system exclusively on clean, well-illuminated images. Identify at least one experimental result that disagrees with theoretical expectations and explain the discrepancy using measured evidence, possible sources of error, and the limitations of the experimental setup.*

Ans:

On clean, well-lit fruit the Budget configuration would have looked acceptable; its failure appeared only with realistic flicker, blur and daylight. Removing the flicker alone raised its accuracy from 50.0 % to 77.8 %. So a test set must include the hard cases the line will meet (touching fruit, glare, cloud, dirt on the lens), otherwise the reported accuracy is too optimistic.

A result that disagrees with theory: the Premium configuration, with more pixels and a finer sampling, was less accurate than Balanced (72.9 % against 83.3 %). Possible sources of error are that the bruise detector was tuned for 0.3125 mm/px and not retuned, that my generator has a single noise model, and that 48 fruit is a small test set (one fruit is about 2 %). The experiment shows the limit of comparing hardware without retuning the algorithm for each option. The general limit of my setup is that simulated images cannot reproduce every real effect, so everything must be confirmed on real fruit.

Q7. *Recommend a suitable industrial inspection configuration by incorporating a fourth option: a smart camera with an onboard AI accelerator running a compact object detector. Compare all four configurations in terms of acquisition and processing latency, total*

system cost, seven-day storage, robustness, deployment complexity, and retraining or rule-maintenance effort.

Ans:

Table 27: Four-way comparison including a smart AI camera (D is reasoning, not tested)

Criterion	A. Budget	B. Balanced	C. Premium	D. Smart AI camera
Acquisition + processing latency	about 76 ms (measured, 2 cores)	about 78 ms (measured)	about 352 ms (measured)	about 20–40 ms on the NPU (vendor-typical, not measured)
System cost	lowest	within ₹1.5 lakh (estimate)	over budget	₹1–2.5 lakh + labelling effort (estimate)
7-day storage	small (JPEG)	about 0.4 TB of crops	larger	only results + selected images
Robustness	poor (flicker, blur)	good with the dome	good but slower	good if trained on varied data
Deployment complexity	low	medium	high	medium (vendor tools)
Maintenance	rules	rules / small k-NN	rules	collect, label and retrain for new defects

A smart camera is attractive for speed, but a learned detector needs thousands of labelled fruit images to be trained and audited, and the unit does not have them yet. I recommend B now and D later, once B has produced a labelled archive.

Final Decision

I choose **B – Balanced. Not A:** it rejected every good fruit (100.0 % false rejects) because the flicker bands and motion blur look like bruises. **Not C:** it was less accurate (72.9 % against 83.3 %), 4.5 times slower and over budget. **B:** the best accuracy of the three (83.3 %), unaffected by the simulated cloud, 78 ms per fruit and within budget. **Limit:** B still missed 20.8 % of defective fruit, so the 2 % limit is not yet met; the bruise detector must be improved and validated on real fruit. **Later:** consider D after thousands of labelled images are collected from B's archive.

Evidence provided: configurations (Table 22), simulation code, measured comparison (Table 23), ablation (Table 24), block diagram with time budget (Figure 12), design decision table (Table 25), applications transfer table (Table 26) and four-way comparison (Table 27).

TASK 10 – Re-apply and Adapt the Imaging Front-End [8 Marks]

A new export buyer in Japan requires detection of fruit-fly stings and internal bruises as small as 0.5 mm, some of which are invisible in normal colour images but visible in near-infrared (NIR). To meet demand, the belt speed doubles to 1.0 m/s (16 fruits per second), the shed roof gets skylights (even stronger daylight changes), and the same edge PC and the same ₹1,50,000 budget must be used. Decisions are still needed before the ejector, now only 0.6 s away.

Technique	Basic idea and key concepts / functions
A. Retain + ROI processing	Keep the Task 9 camera and lighting; shorten exposure, crop to a region of interest and process only the fruit area.
B. Modify sampling and acquisition	Higher-resolution global-shutter camera, shorter strobe, enclosed hood to block daylight, 10/12-bit depth.
C. Redesign sensing (multispectral)	RGB + NIR line-scan (or two-camera) sensing inside an enclosure, with new image formation and calibration.

The Japanese buyer has changed the requirements: the line must now detect 0.5 mm insect stings and bruises that are visible only in near-infrared (NIR), and the belt runs twice as fast (1.0 m/s, 16 fruits/s). In this task I re-evaluate my Task 9 design (B – Balanced) and decide whether to **retain**, **modify** or **redesign** it.

Q1. *Evaluate the impact of the revised inspection requirements on the existing system. Identify the original assumptions concerning defect size, conveyor speed, illumination, spectral band, decision time and data volume that are no longer valid, and justify each change. Assess the expected consequences of retaining the Task 9 front-end without modification.*

Ans:

Table 28: Assumptions of the Task 9 design that are no longer valid

Assumption (Task 9)	New requirement	Consequence
smallest defect 2 mm	0.5 mm stings	0.3125 mm/px gives only 1.6 px per sting, below the Nyquist limit of 2 px
belt 0.5 m/s, 8 fruits/s	1.0 m/s, 16 fruits/s	blur doubles, twice the images, half the time per fruit (62 ms at 16/s)
dome blocks most daylight	skylights, stronger changes	the inspection zone must be fully enclosed

Assumption (Task 9)	New requirement	Consequence
visible light is enough	NIR-only bruises	an RGB camera cannot see them at any resolution
1.2 s to decide	0.6 s to the ejector	still enough time but less margin
about 10 MB/s of data	about 60–80 MB/s	the Gigabit link and the storage come under pressure

If the Task 9 front-end were kept unchanged, it would find **0.0 %** of the stings (Table 30), and it could see a bruise that is invisible in RGB only by luck. The buyer would reject the consignment.

Q2. *Recalculate the spatial resolution (mm/pixel) required to detect a 0.5 mm defect, motion blur at a conveyor speed of 1.0 m/s, the required frame rate or line-scan rate, image bit depth and data acquisition rate. Evaluate the original front-end's performance against these requirements using the same computing platform. Identify the components and design assumptions that remain suitable and justify their continued use.*

Ans:

Table 29: Requirements at 1 m/s

Quantity	Calculation	Value
Pixel size for a 0.5 mm defect	0.5 mm ÷ 3 px	0.17 mm/px
Pixels across 400 mm	400 ÷ 0.167	2400 px (about a 2.5k sensor)
Exposure for ≤ ½ pixel blur at 1 m/s	0.083 mm ÷ 1000 mm/s	83 µs
Line-scan rate	1000 mm/s ÷ 0.167 mm	6000 lines/s
Minimum area frame rate	1000 ÷ (300 − 110)	5.3 fps (16 fps triggered, one per fruit)
Data rate, 5 MP area camera at 16 fps	5.0 MB × 16	80 MB/s
Data rate, RGB + NIR line-scan	2448 × 6000 × 4 bands	59 MB/s

Quantity	Calculation	Value
Bit depth	NIR contrast is low against sensor noise	10-bit for NIR, 8-bit for RGB

Still suitable from Task 9: the grader logic (connectivity, De, circular hue), the calibration and flat-field procedures, the storage policy of fruit crops only, and the edge PC for the fruit-level logic. **Not suitable:** the camera resolution, the visible-only sensing, the exposure, and the processing budget (see Q3).

Q3. *Compare Strategies A, B and C in terms of 0.5 mm defect detection, NIR-only defect detection, decision time, storage requirements, implementation cost and robustness. Use Python/OpenCV simulations, measured results and engineering calculations to substantiate the comparison. Recommend a revised image acquisition and processing front-end, justifying the selected strategy and the modifications required.*

Ans:

I tested three strategies on the same 12 fruit. Each had three stings of 0.5–0.7 mm, and every second fruit had a bruise that is also dark in a simulated NIR channel. **A. Retain + ROI:** keep the Task 9 camera (0.3125 mm/px) and shorten the exposure to 0.25 ms. **B. Modify:** a 5 MP global-shutter camera (0.163 mm/px), 0.05 ms strobe and a full hood. **C. Redesign:** an RGB + NIR line-scan camera at 0.167 mm/px, synchronised with a belt encoder.

Important limit of this test: in my generator the bruise also has a faint visible part (9 grey levels), so the RGB-only results for A and B are an optimistic upper bound. A truly NIR-only defect cannot be seen in RGB at any resolution.

t910.py, Task 10 part (three adaptation strategies)

```
 ----------------
V10 = 1000.0
req = dict(mmpx=0.5 / 3, px=400 / (0.5 / 3), blur_ms=(0.5 / 3) / 2 / V10 * 1000,
line_rate=V10 / (0.5 / 3), minfps=V10 / (300 - 110), mbs_area=2448 * 2048 * 16 /
1e6, mbs_line=2448 * (V10 / (0.5 / 3)) * 4 / 1e6)
strat = {'A Retain + ROI (0.3125 mm/px, 0.25 ms exposure)': dict(mm=0.3125,
blur=V10 * 0.25 / 1000, nir=False),
         'B Modify (0.163 mm/px, 0.05 ms strobe, hood)': dict(mm=0.1634,
blur=V10 * 0.05 / 1000, nir=False),
         'C Redesign (RGB+NIR line-scan, 0.167 mm/px)': dict(mm=0.1667,
blur=0.0, nir=True)}
rows = []
for name, s in strat.items():
    r = np.random.default_rng(101); sd = st = nd = nt = 0; ts = []
    for i in range(12):
        im, gt = mango.render(r, cls='turning', mm_per_px=s['mm'], n_spots=3,
spot_mm=(0.5, 0.7), bruise=(i % 2 == 0), bruise_drop=9.0, nir=True, spec=14,
noise=1.5)
        im = motion_blur(im, s['blur'], s['mm']); t = time.perf_counter()
        f = mango.measure(im, s['mm'], min_spot_mm2=0.1); sd +=
min(len(f['spots']), 3) if f else 0; st += 3
```

```
        if gt['bruise']:
            nt += 1
            if s['nir'] and f:
                n = gt['nir'].astype(np.float32); m = f['inner'] > 0; drop =
cv2.GaussianBlur(np.median(n[m]) - n, (0, 0), 3) * m; nd += int((drop >
20).sum() * s['mm'] ** 2 > 20)
            elif f: nd += int(f['bruise'])
        ts.append((time.perf_counter() - t) * 1000)
    rows.append([name, sd / st, nd / max(nt, 1), float(np.median(ts))])
save('t10', dict(req=req, rows=rows)); print(req, rows)
r = np.random.default_rng(5); im, gt = mango.render(r, cls='turning',
mm_per_px=0.1667, bruise=True, bruise_drop=9.0, nir=True, n_spots=3,
spot_mm=(0.5, 0.7), spec=14)
grid([im, gt['nir']], ['visible (RGB): bruise barely visible', 'NIR channel:
same bruise is dark'], f'{FIG}/t10_nir.png', size=4.4)
```

Table 30: Measured comparison of the three strategies (12 fruit)

Strategy	0.5 mm stings found	Bruises found	ms / fruit (2 cores)	≤ 0.6 s?
A Retain + ROI (0.3125 mm/px, 0.25 ms exposure)	0.0 %	33.3 %	70	yes
B Modify (0.163 mm/px, 0.05 ms strobe, hood)	25.0 %	66.7 %	356	yes
C Redesign (RGB+NIR line-scan, 0.167 mm/px)	16.7 %	100.0 %	354	yes

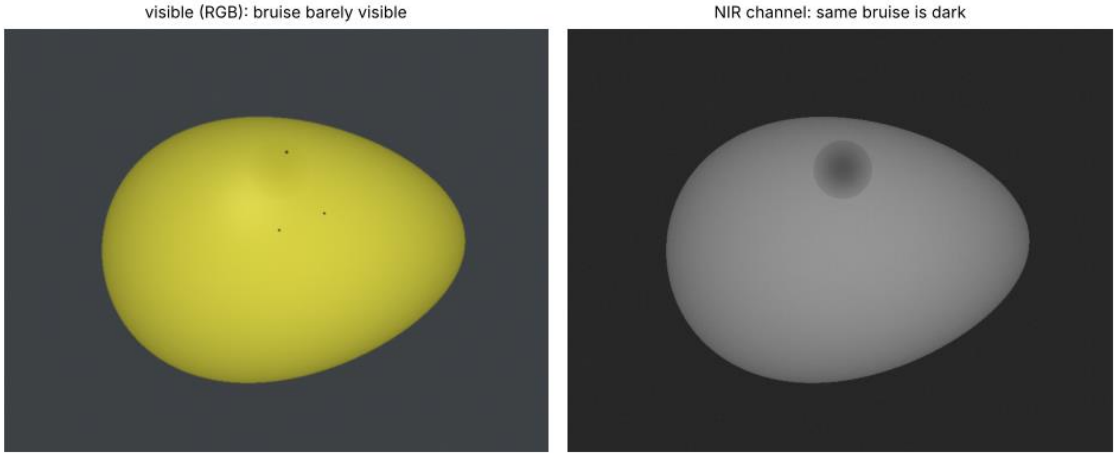


Figure 13: The same bruise in the visible image and in the NIR channel

Table 31: Comparison on the six criteria

Criterion	A. Retain + ROI	B. Modify	C. Redesign
0.5 mm stings	0.0 % found; 1.6 px per sting	25.0 % found; 3 px per sting	16.7 % found; 3 px per sting
NIR-only defects	cannot be seen in RGB (33.3 % of my partly visible bruises)	cannot be seen in RGB (66.7 %)	seen in the NIR channel (100.0 %)
Decision time	70 ms per fruit	356 ms per fruit (2 cores)	354 ms per fruit (2 cores)
Storage	unchanged	about 4 times larger images	about 4 bands, a large data rate (59 MB/s)
Cost (estimate)	no extra cost	5 MP camera + hood	about ₹1.3–1.5 lakh (estimate)
Robustness	fails the new requirement	daylight blocked by hood	enclosed, synchronised with the belt

- **A** fails: at 0.3125 mm/px a 0.5 mm sting covers 1.6 pixels, and the RGB camera cannot see NIR defects.
- **B** found only 25.0 % of the stings in my test, even at 3 pixels per sting, and still cannot see NIR-only defects. My grader's sting detection, not the pixel count alone, is the limit here.
- **C** was the only strategy that found the NIR bruises (100.0 %), but my sting detection was also weak (16.7 %). The stings need a better detector (for example a matched-filter or a learned detector); I did not achieve reliable 0.5 mm detection in this assignment.
- **Processing load:** one fruit took about 354 ms on one core of my 2-core PC. At 16 fruits/s that is about 5.7 core-seconds per second, more than the four cores of the edge PC provide. So the processing must be made cheaper (grade only a fruit ROI, find fruit at low resolution and run the fine detector only inside it) or a faster PC is needed.

Q4. *Justify the final front-end design using calculated performance requirements, Task 9 baseline measurements, experimental results, available budget and computing hardware limitations. Evaluate the trade-offs associated with resolution, acquisition speed, spectral*

capability, processing time and storage. State one specific operating condition under which the proposed design would no longer be suitable and justify the limitation.

Ans:

My recommended front-end is an enclosed tunnel with a 2.5k-pixel RGB + NIR line-scan camera (or two synchronised cameras), running at 6000 lines/s from a belt encoder, with a line light covering visible light and 850 nm. The decision time (354 ms per fruit on one core of my PC) is inside 0.6 s, but the throughput requirement of 16 fruits/s is not yet met on the four-core PC (see the load calculation). **Trade-offs I accept:** more integration work (encoder, NIR calibration), a larger data rate, line-scan geometry that needs constant belt-speed tracking, and a processing cost that needs optimisation. **Operating condition where it would no longer be suitable:** if the fruit roll or bounce on the belt, a line-scan camera sees a distorted fruit and misses defects on the hidden side; then a roller conveyor with views from several sides would be required.

Q5. *Evaluate how few-shot learning with a pretrained foundation model could reduce the number of labelled images required to detect newly introduced NIR-specific defects. Discuss the risks associated with limited training samples, domain shift, false acceptance or rejection, model traceability and reproducibility. Recommend measures to maintain reliable defect classification and provide auditable evidence for export-quality inspection.*

Ans:

I did not run a foundation model, so this part is reasoning. A pretrained model such as DINOv2 or CLIP already has general visual features, so a new NIR defect class could be added from only 10–50 labelled examples by training a small classifier head on its embeddings. **Risks:** so few samples may not cover all bruise ages and varieties; NIR images differ from the RGB photographs the model was trained on (domain shift), so the features may not separate the classes; thresholds chosen on few samples can give unexpected false accepts or rejects; and a model that changes silently between versions is not reproducible. **Measures:** keep a held-out validation set and report confidence intervals; send low-confidence fruit to a review bin; version and log the model for every decision; re-validate monthly on audited fruit; and keep the classical NIR-threshold rule as an auditable cross-check.

Final Decision

I **REDESIGN** the front-end (Strategy C). **Not RETAIN (A):** the Task 9 camera found 0.0 % of the stings, and its assumptions (2 mm defects, visible light only, 0.5 m/s) are no longer valid. **Not only MODIFY (B):** a 5 MP camera gave more pixels but still cannot see NIR-only defects, because the problem is the wavelength and not the number of pixels. **REDESIGN (C):** only the enclosed RGB + NIR front-end sees the NIR bruises (100.0 %), with a decision time of 354 ms per fruit. **Limits:** reliable detection of 0.5 mm stings was not achieved by my grader, and the processing cost at 16 fruits/s exceeds my four-core estimate, so both must be solved (better sting detector, ROI-based processing) before

commissioning. The decision to redesign follows from the evidence on wavelength; the algorithm still needs work.

Evidence provided: impact analysis (Table 28), recalculated requirements (Table 29), measured comparison of the three strategies (Tables 30 and 31, Figure 13), justification with limits, and the few-shot learning discussion.

MARK DISTRIBUTION

Task	Marks
Task 1 – Overview and Features of Computer Vision	4
Task 2 – Levels of Computer Vision and Image Processing	5
Task 3 – Digital Image Fundamentals I	4
Task 4 – Digital Image Fundamentals II	4
Task 5 – Image Sensing and Acquisition	5
Task 6 – Sampling	5
Task 7 – Quantization	4
Task 8 – Image Formation Models	5
Task 9 – Applications and the Integrated System	6
Task 10 – Re-apply and Adapt the Imaging Front-End	8
TOTAL	50